Computers in Industry 150 (2023) 103945

Contents lists available at ScienceDirect

Computers in Industry

journal homepage: www.sciencedirect.com/journal/computers-in-industry

Semi-automatic representation of design code based on knowledge graph for automated compliance checking

Mingsong Yang [a], Qin Zhao [b], Lei Zhu [a], Haining Meng [a], Kehai Chen [c], Zongjian Li [d], Xinhong Hei [a,e,*]

[a] School of Computer Science and Engineering, Xi'an University of Technology, Xi'an 710048, Shaanxi, China
[b] School of Civil Engineering and Architecture, Xi'an University of Technology, Xi'an 710048, Shaanxi, China
[c] School of Computer Science and Technology, Harbin Institute of Technology, Shenzhen 518055, Guangdong, China
[d] State Key Laboratory of Rail Transit Engineering Informatization (FSDI), Xi'an 710043, Shaanxi, China
[e] Shaanxi Key Laboratory of Network Computing and Security Technology (Xi'an University of Technology), Xi'an 710048, Shaanxi, China

ARTICLE INFO

Keywords:
Automated compliance checking (ACC)
Design code representation
Knowledge graph (KG)
Building information model (BIM)
Building design

ABSTRACT

Automated compliance checking (ACC) intends to verify the compliance of designs in construction industry by design codes. The ability to interpret and represent semantic information of design codes determines the maximum application scope of ACC. However, design codes are clause texts written in natural languages and existing ACC studies usually use relatively low-complexity code clause samples. At present, the lack of an accurate representation model for design codes leads to difficulties in representing the implicit information, nested logic, and complex relations contained in high-complexity clauses in codes. To address this problem, this research establishes a new representation model based on knowledge graph (KG). Four schemas are proposed into the model including order, complex, event and integration schemas. Further, an accompanying methodology for semi-automatic construction of design code KG (DCKG) is proposed. It includes four parts: interpretation, reconstruction, organization, and implementation. Where the implementation part develops a code annotation platform. In the case study and experiment, a scenario of checking a building information model (BIM) of metro station by *GB50157–2013 Code for Design of Metro* is adopted to validate the newly proposed representation model and the automated compliance process. The results show that the proposed model and method are correct and feasible, and our model outperforms other models in the representation ability of design codes.

1. Introduction

Compliant building design is the basis and guarantee for construction, operation and maintenance (Amor and Dimyadi, 2021). Failure to timely identify omissions and errors in the design will probably cause design changes or re-work (Lee et al., 2020). And these unchecked design hazards may even threaten the lives and property of citizens, especially in infrastructure projects. The traditional compliance checking largely depends on the individual competence of the reviewer. ACC is expected to reduce manual review errors, time, cost, and labor (Tan et al., 2010). Eastman et al. reviewed the early research of ACC and proposed that "*the field of rule checking in building is just emerging*" (Eastman et al., 2009). Since then, ACC research has gradually improved (Pauwels et al., 2011; Schwabe et al., 2019). The core of ACC research can be divided into four parts according to its implementation process: 1) Interpretation and representation of design codes; 2) Enrichment and preparation of design BIM models; 3) Execution and judgment of compliance checking, and 4) Interpretation and feedback of checking results.

Beach et al. launched a survey on industry attitudes toward ACC (Beach et al., 2020). The highest average rating for barriers to ACC was "*Lack of shared open standards for regulation clauses*", and 1/3 of respondents also identified "*Lack of precise digitisable regulations*" as a key barrier. In fact, the essence of these barriers is the inherent complexities of code clauses (Solihin and Eastman, 2015). However, Complex clauses with multiple requirements are common in design codes but are rarely mentioned in relevant research (Zhou et al., 2022). Existing methods usually use a sampling of relatively low-complexity code clauses or manually deal with a limited range of code clauses in some specific areas. Such as hand-selected clauses or pre-deconstructed subclauses

* Corresponding author at: School of Computer Science and Engineering, Xi'an University of Technology, Xi'an 710048, Shaanxi, China.
E-mail address: heixinhong@xaut.edu.cn (X. Hei).

https://doi.org/10.1016/j.compind.2023.103945
Received 19 January 2023; Received in revised form 9 April 2023; Accepted 9 May 2023
Available online 23 May 2023

containing only binary semantics. In addition, some studies have also implemented automated code clauses processing based on NLP (Xue and Zhang, 2021; Zhou et al., 2022), but the structural diversity and semantic complexity of code clauses are still underestimated or ignored. Therefore, ensuring the complete representation of semantic information in high-complexity clauses is a key issue in design code representation.

KG will become one of the important tools for knowledge management in the architecture, engineering and construction (AEC) industry in the future (Deng et al., 2022). KG is essentially a semantic network where the nodes represent entities or concepts and the edges represent various semantic relations between entities/concepts (Paulheim, 2016). After Google first proposed the KG to enhance user search experience (Singhal, 2012), various KGs were continuously developed and applied (Ehrlinger and Wöß, 2016; Noy et al., 2019). KGs are further classified into Generic-KG (GKG) and Domain-KG (DKG) according to application scenarios and knowledge requirements (Abu-Salih, 2021; Lin et al., 2021). Among them, GKG focuses on general domain encyclopedic knowledge, and DKG focuses on modeling relations and entities in a specific domain (Abu-Salih, 2021). DKGs bring flexibility, accuracy and interpretability advantages to the domain knowledge representation (Hogan et al., 2021; Li et al., 2021). Developing a design code knowledge graph (DCKG) is expected to bring new enhancements to ACC (Hu et al., 2022). However, most DKG lack a high-quality corpus for automatic construction compared to the GKG with massive data sources.

Basically, the construction of DKG is usually driven by specific downstream applications (Gentile et al., 2019; Huang et al., 2020; Ma, 2022). In particular, domain knowledge for some application scenarios is strict, e.g., legislation-related knowledge, contraindications of drugs, etc. The interpretation and representation of such knowledge are not allowed to be missing, ambiguous or mistaken, and these DKG require higher representation ability and deeper knowledge level (Lin et al., 2021). The same is true for DCKG used in ACC. Furthermore, the DKG construction methods vary for different domain applications (Abu-Salih, 2021). Human expert participation is necessary to guarantee the quality of DKG (Kejriwal, 2019; Zhang et al., 2020). The key issue for high-quality DCKG construction is how and in which steps domain experts are involved.

To address above issues, this research establishes a design code representation model based on KG, which can accurately represent knowledge contained in high-complexity code clauses. In addition, we contribute an accompanying methodology for semi-automatic DCKG construction, which clarifies the interpretation and reconstruction of code clauses, and takes full advantage of the proposed model in design code representation. In addition, a design code annotation platform has been developed to implement the DCKG construction.

The remainder of this paper is organized as follows. Section 2 reviews the related work and highlights research gaps. Section 3 focuses on the new representation model of design code. Section 4 elaborates on the accompanying methodology. Section 5 illustrates and analyzes the results of the experiment and case study. Section 6 discusses the contributions of this research and notes the limitations. Section 7 concludes this research.

2. Relate work

The interpretation and representation of design codes aim to represent the code clauses in a computer-readable manner without omitting or changing the meaning of original clause texts (Zhang et al., 2022). The ability to interpret and represent design codes determines the maximum application scope of ACC. In 1966, Fenves used tabular decision logic to represent design code for auxiliary design and review, which was the first attempt at ACC research (Fenves, 1966). Afterward, the researchers used different knowledge representation models to complete the representation of design code for ACC (Eastman et al., 2009; Lee et al., 2020; Pauwels and Zhang, 2015). This section overviews the main models of interpretation and representation used for ACC, and compares their representation ability for design codes. Also, the KG is analyzed.

2.1. Interpretation and representation model

2.1.1. Commercial software model

ACC commercial software can be divided into two types. One is the independent checking software. The other is the design software checking plug-in. Solibri Model Checker (SMC), as a representative product of independent checking software, interprets and represents design code knowledge through parameterized table structures and rule sets (Solibri, 2021). SMARTreview Automated Plan Review (APR)[1] is a checking plug-in developed for Revit (Clayton et al., 2013), which is provided to help designers make their projects comply with the design code according to the proprietary internal rule engine. In fact, both types of mainstream commercial checking software are hard-coded rule bases. In addition, the creation and maintenance of hard-coded rule bases rely heavily on the collaborative development of domain experts and programmers. Also, the built-in environment of the rule base and plug-ins make it challenging to update the knowledge synchronously with the modification of design codes.

2.1.2. Logic-based model

Kerrigan et al. proposed a code clauses representation model based on first-order logic (FOL) and constructed semantic tags for logical relation representation and flow control based on XML framework, to improve the operability of the design code representation model (Kerrigan and Law, 2005). Zhang and El-Gohary combined FOL with ontology for code representation and compliance reasoning (Zhang and El-Gohary, 2017). Later, their team also proposed automated interpretation methods, such as NLP-based clauses information extraction (R. Zhang and El-Gohary, 2021) and part-of-speech tagging (Xue and Zhang, 2021) for code clauses based on deep learning. Nawari proposed a generic adaptive framework to transform the code clauses into computable representation based on FOL. And six concepts of content, provisional, dependency, ambiguity, exceptions and alternatives are defined to mark and interpret code clauses (Nawari, 2019). Solihin and Eastman use a FOL-compliant concept graph to interpret and represent the code clauses (Solihin and Eastman, 2016). The logic-based model has a formal grammatical structure with a more fixed pattern consistent with natural language. However, there are limitations in interpreting and representing complex code clauses, such as high cost of logical rules construction, selection and reasoning, and poor flexibility in design code representation.

2.1.3. Object-oriented model

Yang proposed a hybrid object-based model considering that both logic-based and generative rule-based representations do not involve the underlying object model (Yang and Xu, 2004). Afterward, in collaboration with the National Building Code Organization of England and Wales, Malsane et al. developed an object-oriented model for code clauses representation with well-defined objects and properties (Malsane et al., 2015). Balaban et al. developed the Fire Codes Checker Platform for Turkish fire codes (Balaban et al., 2012), relying on an object-oriented engine provided by Express Data Manager. The object-oriented models have improved the interoperability and maintainability of design code representation. However, the representation ability of such models is limited by the initial setting of object classes, since the design codes are interpreted and represented through the object classes and their hierarchies.

[1] SMARTreview APR: https://www.smartreview.biz/home.

2.1.4. Semantic web-based model

Beach et al. proposed a semantic framework for design codes containing three T-box ontologies and represented as SWRL rules by adding XML tags to the code clauses (Beach et al., 2015). Zhong's team proposed ontology-based check frameworks for constructing quality compliance checking (Zhong et al., 2012) and building environment monitoring (Zhong et al., 2018). Xu and Cai proposed an ontology and rule-based approach for interpreting code clauses, where the *spatialConfiguration* class is defined to describe the main objects in a spatial constraint and the *inConctionWith* property is defined to organize the different *spatialConfigurations*. After that, seven clauses are extracted from the utility regulations to be checked with the SPARQL query (Xu and Cai, 2020). Jiang et al. transformed clauses into T-box and R-box directly combined with the constructed ontologies and rule sets for ACC (Jiang et al., 2022). Semantic web-based models represent domain knowledge as concepts, instances, and the relations among them. However, compatible rule language or query language is also needed to provide logical support in representing design codes, since Web Ontology Language (OWL) cannot represent generative rules. It requires the deep involvement of domain experts for both the statement formulation of complex rules or queries and the range delineating of code ontology for ACC.

2.1.5. New modeling languages

Some studies have proposed new modeling languages to interpret and represent design code. Hjelseth and Nisbet interpret the code clauses into four parts, namely Requirement, Applicability, Selection and Exceptions (RASE), and represent them through semantic labels (Hjelseth and Nisbet, 2010). Macit İlal and Günaydın merged the RASE method and four-layer framework to proposed a new code representation that separately modeled the domain concept, rule, ruleset, and code organization (Macit İlal and Günaydın, 2017). Schwabe et al. used the Drools language to interpret and represent the code clauses (Schwabe et al., 2019), combined with the four-level classification (Solihin and Eastman, 2015) and RASE semantic tags. Zhou et al. proposed an interpretation framework based on context-free grammar, which parses the code clause into a tree structure (Zhou et al., 2022). Modeling code clauses with semantic tags or open standards prefer a well-defined structure of the clause text. However, the text has variable syntactic structures and complex semantic types, which makes it difficult to ensure the consistency and non-redundancy of the interpretation and representation of design codes.

2.1.6. Visual programming-based model

Visual programming languages (VPL) are noticed, given that most

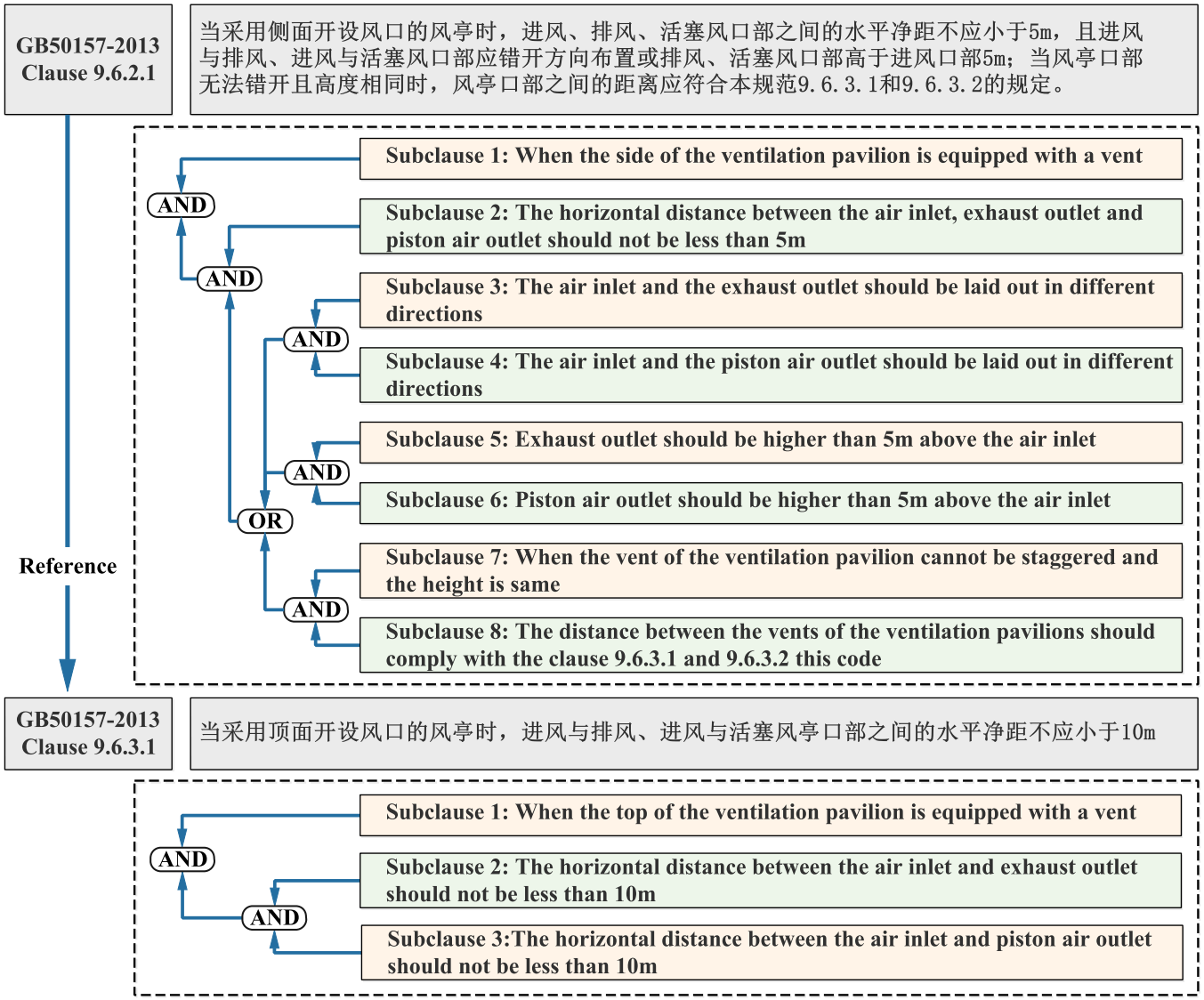

Fig. 1. Disassembly of complex code clauses.

AEC practitioners lack programming experience. Kim et al. proposed KBim Visual Language (Kim et al., 2019) based on KBimCode to interpret the design codes of Korean using visual symbols. Ghannad et al. proposed a modular framework for code interpretation and representation combining the VPL and open standards LegalRuleML (Ghannad et al., 2019). Häußler et al. combined two models and notations to visualize the railway design codes (Häußler et al., 2021). VPL-based representations are easier to understand, more friendly for designers and domain experts, and better applied in ACC for specific domains or checkpoints. However, it faces the same problem as object-oriented representation because the lack of class and event definitions in the code representation will cause illogical structure and confusing relationships.

2.2. Comparative analysis

Analysis of the semantic types in design codes is necessary to judge the model representation ability (Zhang et al., 2022). A complex clause containing multiple semantic types is selected from the *GB50157–2013 Code for Design of Metro*, As shown in Fig. 1.

Clause 9.6.2.1 is divided into eight subclauses and some semantics at the clause level are found, including the association relations and logical relations between subclauses. In addition, there are code level semantic types, such as reference relations between clauses and hierarchical organization of codes. And the subclause level semantic types can be divided according to specific semantic relations as follows:

- Unitary semantic (ignoring context) (e.g., existence)
- Unitary semantic (considering context) (e.g., side of the ventilation pavilion)
- Binary semantic (numerical) (e.g., Height > 10m)
- Binary semantic (non-numeric) (e.g., contains, property)
- Nested semantic (e.g., the distance between A and B is less than C)
- Multiple semantic (e.g., A offers B to C)

According to the above partition, Table 1 shows a comparative analysis of the typical models reviewed in Section 2.1 applied to design codes. In the table, '√' indicates that the model supports current item, and '×' indicates that the model is not supported or elaborated in the relevant study.

The researchers used various models to accomplish the design code interpretation and representation. All models listed in Table 1 can represent unitary semantics (ignoring context), binary semantics and the association relations between subclauses. However, fewer can accurately represent complex semantic types, such as only two considering multiple semantics or hierarchical organization of codes. In particular, no model can simultaneously represent all the semantic types.

2.3. Knowledge graph

KG is a human-recognizable, machine-friendly knowledge representation model (Ehrlinger and Wöß, 2016). It organizes data based on the directed attribute graph structure, which effectively reduces the complexity of representation model and provides greater flexibility and interpretability (Hogan et al., 2021). As the basic semantic structure in KG, the triple can accurately represent the binary semantic for both the numerical and non-numerical classes in the code clauses (Wang et al., 2020). However, the triple simplifies the complexity of actual semantic information.

It is found that there are more than 33.3 % of entities in the Freebase KG participate in non-binary relations (Wen et al., 2016). In addition, 61 % of relations in the Freebase KG are observed to be non-binary (Fatemi et al., 2020). The directed graph-structure-based data organization facilitates the extension of KG model. Wang et al. extended the triple in KG by adding additional attribute-value sets to represent the *n*-ary relations (Wang et al., 2021). Li et al. found that the attributes of the triples are useful for applying the medical KG, so they extended the triples to quadruples with additional fields to store information that cannot be represented by triples (Li et al., 2020). These extended KGs enhance the ability to represent complex semantics.

Table 1
Comparison of representation models.

Model	Literature	Subclause level						Clause level		Code level	
		Unitary semantic		Binary semantic		Multiple semantic	Nested semantic	Relations between subclauses	Relations between subclause sets	Reference relation between clauses	Hierarchical organization of codes
		Ignoring context	Considering context	Numerical	Non-numeric						
Commercial software model	(Solibri, 2021)	√	×	√	√	×	×	√	×	×	×
	(SMARTreview, n.d.)	√	×	√	√	×	×	√	×	×	×
Logic-based model	(J.Zhang and El-Gohary, 2017)	√	×	√	√	√	√	√	×	×	×
	(Nawari, 2019)	√	×	√	√	√	√	√	√	×	×
Object-oriented model	(Malsane et al., 2015)	√	×	√	√	×	√	√	×	×	×
	(Balaban et al., 2012)	√	×	√	√	×	√	√	×	×	×
Semantic web-based model	(B.Zhong et al., 2018)	√	×	√	√	×	×	√	√	×	×
	(Jiang et al., 2022)	√	×	√	√	×	√	√	√	×	×
New modeling languages	(Hjelseth and Nisbet, 2010)	√	×	√	√	×	×	√	×	×	×
	(Zhou et al., 2022)	√	×	√	√	×	×	√	√	×	×
	(Macit İlal and Günaydın, 2017)	√	×	√	√	×	×	√	√	√	√
Visual languages	(Kim et al., 2019)	√	×	√	√	×	×	√	√	×	×
	(Häußler et al., 2021)	√	×	√	√	×	×	√	×	×	×
	(Ghannad et al., 2019)	√	√	√	√	×	×	√	×	×	×

There are two ways to construct KG. One is automatic construction methods such as direct mapping for structured data (Stoica et al., 2019), wrappers for semi-structured data (Nie et al., 2019), and information extraction for unstructured data (Pingle et al., 2019). The other method is semi-automatically constructed, soliciting direct contributions from domain experts through customized collaborative-editing platforms. Kondreddi et al. developed the HIGGINS platform for semi-automatic construction (Kondreddi et al., 2013). Zhang et al. proposed the semi-auto-constructed healthcare KG framework and developed a platform to assist KG construction, incorporating clinicians' knowledge into health KG (Zhang et al., 2020). Tang et al. developed a semantic annotation platform for building high-quality legal KG (Tang et al., 2020). The participation of human experts will guarantee the high quality of KG (Paulheim, 2018). Stakeholder participation in the development process of DKG will facilitate the full exploit KG's advantages in knowledge representation and interpretation (Li et al., 2021).

3. Design code representation model based on KG

Fig. 2 illustrates the design code representation model based on KG. Four schemas are proposed to ensure that the DCKG can represent all semantic types at the subclause, clause, and code levels completely and accurately. We propose the order schema and complex schema to represent semantic information at the subclause level. The event schema is proposed to complete the representation of implicit semantics and logic at the clause level. We also propose the integration schema to improve the semantic model and organization at the code level.

3.1. Representation schemas for subclause level

The subclause level semantic types include unitary, binary, multiple and nested semantics. The binary semantics can be represented by triples only, such as <*Subject*, *Predicate*, *Object*>, <*Object*, *Property*, *Value*> and <*Object*, *Comparator*, *Value*>, where the comparator is a relational operator (e.g., >, <, =, etc.) (Macit İlal and Günaydın, 2017). We proposed the order and complex schema for other subclause level semantic types.

3.1.1. Order schema

It is a common semantic type in subclauses to impose unitary constraints on building elements in quantity, sequence, position, etc. Existing KGs treat the unitary semantic as self-referential relation of entity and represent as a triple <*A*, *r1*, *A*>. As shown in Fig. 3, the triple <*A*, *r1*, *A*> in KG can accurately represent the actual semantic when the unitary semantic exists independently, i.e., no context information or can be ignored.

When an entity with unitary semantic has other constraints in the same subclause, the representation will depend on the unitary semantic, and the semantic order information implied in the context. As shown in Fig. 4, the subclause path is the representation obtained from triples fusion of self-referential relations and other constraints. The actual semantic of the subclause path is one of four cases listed: *Forward, Backward, Outward* and *Inward*.

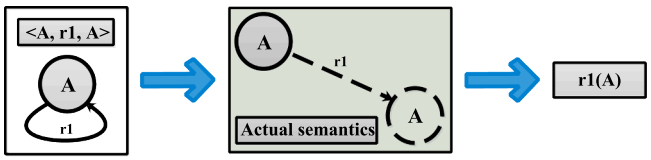

Fig. 3. Expression of unitary semantic (ignoring context).

Specifically, the "*Forward*" indicates that self-referential relation *r2* is included in the forward relation *r1* of node *B*, equivalent to *r1*(*A*, *r2*(*B*)) ∩*r3*(*B*, *C*). The "*Backward*" indicates that self-referential relation *r2* is included in the backward relation *r3* of node *B*, equivalent to *r1*(*A*, *B*) ∩*r3*(*r2*(*B*), *C*). The "*Inward*" indicates that self-referential relation *r2* is included in both the relation *r1* and *r3* of node *B*, equivalent to *r1*(*A*, *r2*(*B*)) ∩*r3*(*r2*(*B*), *C*). The "*Outward*" indicates that self-referential relation *r2* is independent of the relation *r1* and *r3*, equivalent to *r1*(*A*, *B*) ∩*r2*(*B*) ∩*r3*(*B*, *C*).

The checking execution requires querying and merging instances that satisfy different triples in the subclause path. Assume that the instances satisfying triples in the path of Fig. 4 are given in Table 2. Then, the executions order and count among four different cases are shown in Table 3.

It can be seen that different actual semantics have different orders and counts of checking executions. If the representation of design code ignores the semantic order ambiguity brought by the context information of unitary semantic, it will affect the execution order and time required for checking. It is possible to return incorrect checking results for instances different from those in Table 2.

Therefore, an order schema is proposed to enhance the representation of unitary semantic with context information. As shown in Fig. 5, an auxiliary node "*Order Node*" is added to constrain the self-referential relation. Then, an attribute triple <*Order Node*, *E_ID*, *value*> is added to the order node to set a one-to-one link between the auxiliary node and self-referential edge. Finally, the semantic orders are distinguished by the newly added directed edges, which point to the node with the self-referential edge. The label of edge takes *Forward*, *Backward*, *Outward*, or *Inward*.

3.1.2. Complex schema

There are also nested semantics and multiple semantics in subclauses, which can be formalized as *r1*(*r2*(*A*, *B*), *C*) and *r1*(*A*, *B*, *C*) by the predicate function. The triples in KG can represent binary or unitary semantics by a directed edge with head and tail nodes, but they cannot correctly represent nested semantics and multiple semantics.

Wang et al. proposed an representation for n-ary relations in KGs (Wang et al., 2021), mainly by adding additional attribute-value sets {(*attribute_i*, *value_i*): *i*=*1*, ., *m*} to triples. Also, to preserve the complete semantics into a graph, they treated relations, entities, attributes and values as nodes and introduced edges between four node types. However, the above approach cannot determine which entities in multiple semantic should be added as an additional attribute-value set, and which belong to the triple. For example, entities in the sentence "*Entity_1 sends Entity_2 to Entity_3*".

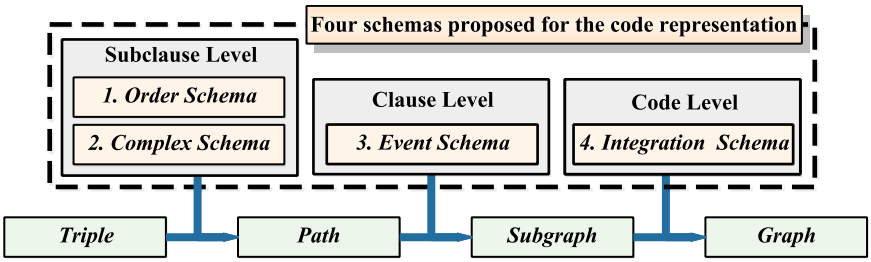

Fig. 2. Design code representation model based on KG.

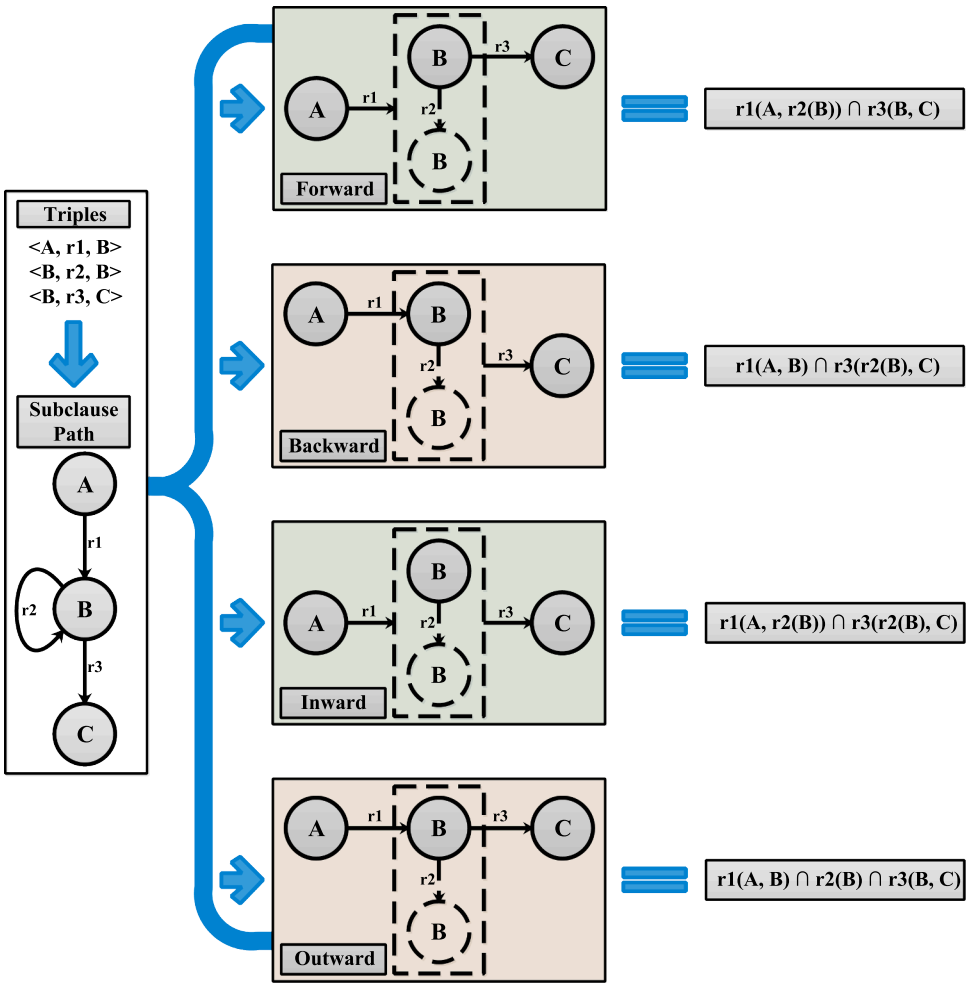

Fig. 4. Expression of unitary semantic (considering context).

Table 2
Assumed instances of triples.

Entities and relations	Count	Instances
A	10	a1, a2, a3, a4, a5, a6, a7, a8, a9, a10
B	10	b1, b2, b3, b4, b5, b6, b7, b8, b9, b10
C	10	c1, c2, c3, c4, c5, c6, c7, c8, c9, c10
r1(A, B)	4	<a1, b1>, <a2, b2>, <a5, b5>, <a8, b8>
r2(B)	3	b1, b2, b3
r3(B, C)	4	<b1, c1>, <b2, c2>, <b3, c3>, <b6, c6>
r1(A, r2(B))	2	<a1, b1>, <a2, b2>
r3(r2(B), C)	3	<b1, c1>, <b2, c2>, <b3, c3>

Inspired by Wang et al.'s operation of treating relation as node, we propose the complex schema. As shown in Fig. 6, the nested semantic and multiple semantic are abstracted into "*Complex Node*", while directed edges are later created based on the semantic in subclause. The values of edge labels follow the sequential position of the entities in natural language representation.

As shown in the top left corner of Fig. 6, the original multiple

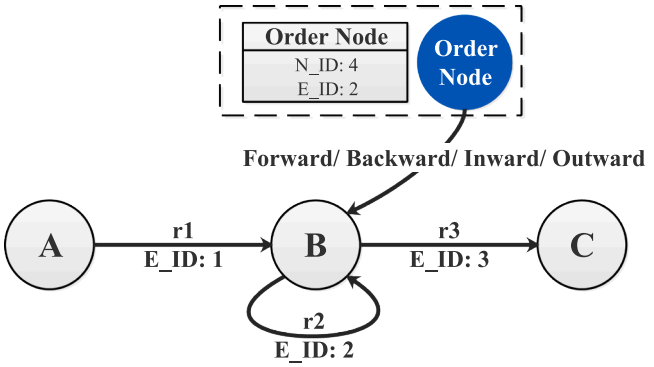

Fig. 5. Representation of order schema.

semantics can be represented as a hypergraph structure, which has an edge that associates multiple nodes, and this edge cannot be represented based on a triple structure in DCKG. Our complex schema abstracts the multiple relation into complex node and combines the entity location

Table 3
Order and count of executions for different actual semantics.

Actual semantics	Execution Order	Count of r1 executions	Count of r2 executions	Count of r3 executions	Count of ∩ executions	Total
Forward	(r2→r1)→r3	30	10	100	8	148
Backward	r1→(r2→r3)	100	10	30	12	152
Inward	r2→r1→r3	30	10	30	6	76
Outward	r1→r2→r3	100	10	100	20	230

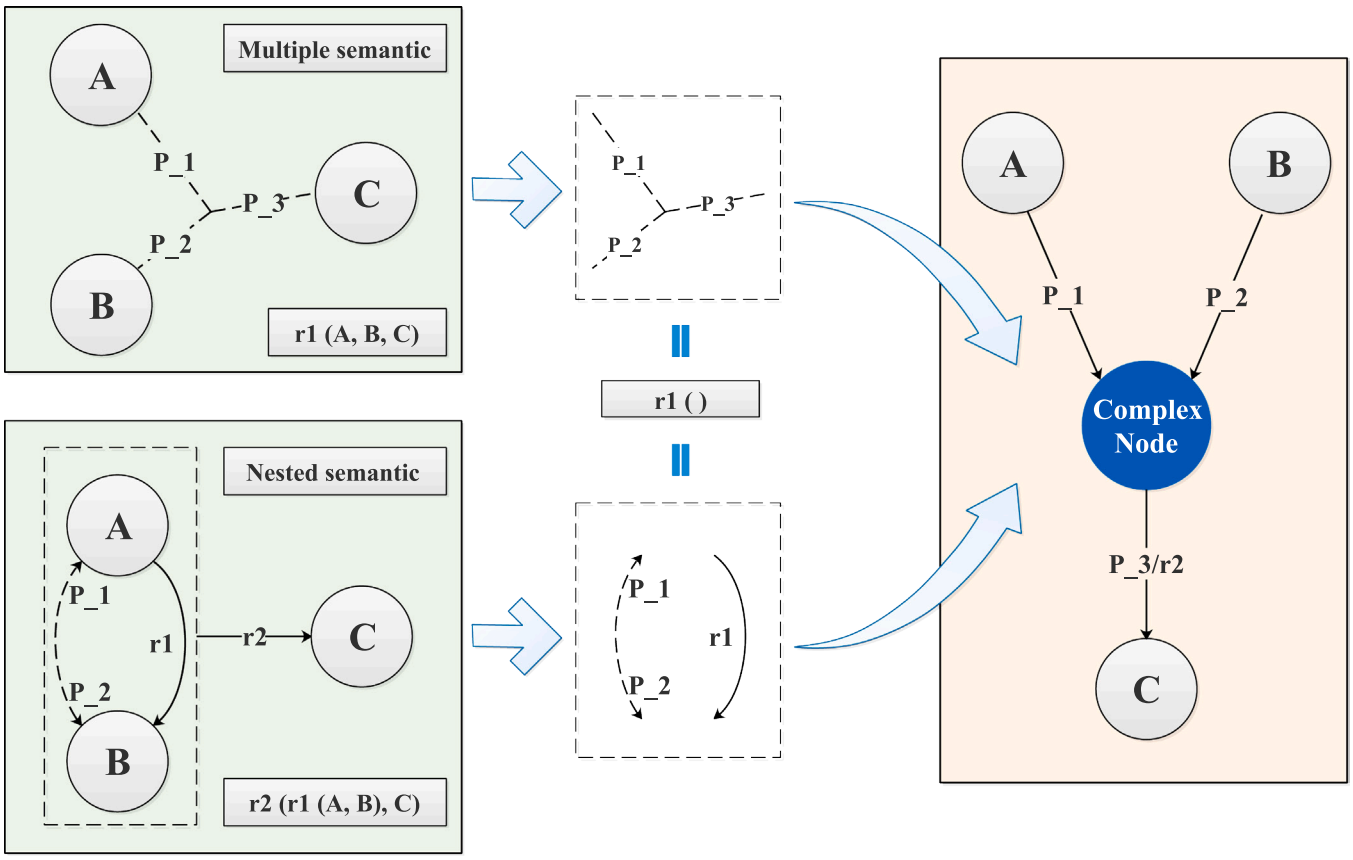

Fig. 6. Representation of complex schema.

information (*P_1*, *P_2*, and *P_3*) in the original multiple semantics to establish the edges between different entities and complex nodes.

As shown in the lower left corner of Fig. 6, the original nested semantics can be represented as an RDF-star graph structure, <<<*A*, *r1*, *B*>>, *r2*, *C*>, where <<*A*, *r1*, *B*>> is the quoted triple in the nested semantics, and <<<*A*, *r1*, *B*>>, *r2*, *C*> is the asserted triple (Arndt et al., 2021). Our complex schema abstracts the r1 relation in the quoted triple as a complex node, and combines the entity location information (*P_1*, *P_2*) in the nested semantics to establish the edges between different entities and complex node.

The pointed edge of complex nodes can discriminate the two semantic types contained in complex schema. The complex schema represented nested semantic *r1*(*ComplexNode*(*A*, *B*), *C*) when the label of pointed edge is a concrete predicate function. The complex schema represented multiple semantic *ComplexNode*(*A*, *B*, *C*) when the label of pointed edge is an entity location.

3.2. Representation schema for clause level – event schema

Each clause of design code consists of at least one subclause. The clause is represented as a subgraph in the KG-based representation model by fusing the same triple in different subclause paths. However, the semantic information represented by a clause subgraph is larger than the actual clause text semantics.

Further analysis reveals the essence. First, all path combinations of the clause subgraph have only some combinations that can correctly represent the actual semantics of original clause text. As shown in Fig. 7, the actual semantics of clause are assumed to be path combination {<*A*→*G*>, <*A*→*C*→*D*→*F*>, <*B*→*C*→*D*→*E*>}. However, the semantics of subclause paths <*A*→*C*→*D*→*E*>and <*B*→*C*→*D*→*F*> are generated incorrectly in the clause subgraph. This is due to merging duplicate triples or nodes in subclause paths.

The second is that the semantic information at the clause level is not represented, and there are implicit relations and logic in clause text. As shown in Fig. 8, assuming that the original clause text implies the information "(*IF* <*A*→*C*→*D*→*E*>*THEN* <*A*→*G*>) *OR* (*IF* <*B*→*C*→*D*→*F*> *THEN* <*A*→*G*>)". However, the current clause subgraphs cannot represent the association relations between subclause paths and the logical relations between path combinations. This is because the subclause paths contained in the clause subgraph are disordered and irrelevant.

In our study, we add precise constraints to the clause subgraphs by the proposed event schema. As shown in Fig. 9, creating the auxiliary node "*Event Node*" and then adding directed edges between the tail nodes of the subclause paths and event node to represent the implicit association relations between different subclause paths in the same path combination. There are two types of these edges: conditional edge "*C_n*" and outcome edge "*O_n*". In addition, bidirectional edges with "OR, AND or XOR" labels between events are added to represent logical relations at the clause level. Unlike the Event KG (Gottschalk and Demidova, 2018), our proposed event schema represents the relations within and between events at a finer granularity.

The event schema can assist the clause subgraph in fully representing the implicit semantics and logic. Nevertheless, the correctness of the clause subgraph representation still needs to be guaranteed because it is required to identify the subclause paths with correct semantics in the event schema. In our study, as shown in Fig. 10, we record the subclause paths containing correct semantics as attribute triples for event nodes. The attribute triple <*EventNode_2*, *C_1*, [*4*, *2*, *5*]> is added to *EventNode_2*, meaning that the "*C_1*" subclause path of this event can be quickly located by the three edges with *E_ID*={*4*, *2*, *5*} in Fig. 9. Such an attribute triple eliminates the ambiguity caused by path fusion in the clause subgraph and accurately represents the original clause text. In addition, we add the attribute triple <*EventNode_2*, *Clause_ID*, *value*> to

the event node to record the code clause that the current event belongs to.

3.3. Representation schema proposed for code level – integration Schema

The semantic types at the code level mainly include the organization of codes and the reference relation between clauses. Design codes are currently organized according to the hierarchical structure "*codes-chapters-sections-clauses*". This natural method of organization is a human-oriented model that makes it easy for designers to understand and query clauses as they work. However, this hierarchical organization hinders the connection between different clauses or codes. In ACC tasks, a specific checkpoint may need to be checked jointly based on multiple clauses in different codes to obtain accurate results. As shown in Fig. 11, two national codes, *GB50157–2013 Code for Design of Metro* and *GB51298–2018 Standard for Fire Protection Design of Metro*, address the requirements for the design of fire compartmentation in metro stations. Moreover, allowing alternative organization methods to exist simultaneously is beneficial (Macit İlal and Günaydın, 2017).

In clause subgraphs, the actual semantics of original clause is represented as one or more events. These events have the same subject matter but are formally independent. The integration schema we propose is an event-based organization of design code. As shown in Fig. 12, it organizes the event nodes according to *Check objects* and *Checkpoints*, and incorporates existing hierarchical organization methods. The integration schema will be the basis for the DCKG schema layer. And the creation of *Check objects* and *Checkpoints* is described in Section 4.4.

In addition, the reference relation between clauses can be further refined into references between different events contained in multiple clauses. Fig. 13 shows the reference relation between the clauses exemplified in Fig. 1. The reference relation is represented by a directed edge with *"Reference"* label between the event nodes from different clause subgraphs. And the edge will make it more transparent, which and what the reference is issued to.

3.4. Design code knowledge graph model (DCKG model)

Combining the above four proposed schemas, the DCKG model can be represented as.

DCKG =(N, R, AN, AR, S).

Where $\boldsymbol{N}=\{n_1, n_2, \ldots, n_i\}$, representing a collection of *i* different entities in the knowledge graph; $\boldsymbol{R}=\{r_1, r_2, \ldots, r_j\}$, representing a collection of j different relations between the i entities; $\boldsymbol{AN}=\{on_1, on_2, \ldots, on_k, cn_1, cn_2, \ldots, cn_m, en_1, en_2, \ldots, en_o, in_1, in_2, \ldots, in_p\}$, represents a collection of auxiliary nodes, containing *k* order nodes, *m* complex nodes, *o* event nodes, and *p* integration nodes; $\boldsymbol{AR}=\{Backward, Forward, Inward, Outward, \ldots, P_1, P_2, \ldots, P_n, OR, AND, XOR, C_1, C_2, \ldots, C_q, O_1, O_2, \ldots, O_r, Contain\}$, representing a collection of newly added relations that accompany the auxiliary nodes in the proposed schemas; $\boldsymbol{S}=\{\boldsymbol{BS}, \boldsymbol{OS}, \boldsymbol{CS}, \boldsymbol{ES}, \boldsymbol{IS}\}$, representing a collection of triples, containing basic schema $\boldsymbol{BS}\subseteq\boldsymbol{N}\times\boldsymbol{R}\times\boldsymbol{E}$, order schema (Section 3.1.1) $\boldsymbol{OS}\subseteq\{on_1, on_2, \ldots, on_k\}\times\{Backward, Forward, Inward, Outward\}\times\boldsymbol{N}$, complex schema (Section 3.1.2) $\boldsymbol{CS}\subseteq\boldsymbol{N}\times\{P_1, P_2, \ldots, P_n\}\times\{cn_1, cn_2, \ldots, cn_m\}\cup\{cn_1, cn_2, \ldots, cn_m\}\times\{P_1, P_2, \ldots, P_n\}\times\boldsymbol{N}\cup\{cn_1, cn_2, \ldots, cn_m\}\times\boldsymbol{R}\times\boldsymbol{N}$, event schema (Section 3.2) $\boldsymbol{ES}\subseteq\boldsymbol{N}\times\{C_1, C_2, \ldots, C_q, O_1, O_2, \ldots, O_r\}\times\{en_1, en_2, \ldots, en_o\}\cup\{en_1, en_2, \ldots, en_o\}\times\{OR, AND, XOR\}\times\{en_1, en_2, \ldots, en_o\}$, integration schema (Section 3.3) $\boldsymbol{IS}\subseteq\{in_1, in_2, \ldots, in_p\}\times Contain\times\{in_1, in_2, \ldots, in_p\}\cup\{in_1, in_2, \ldots, in_p\}\times Contain\times\{en_1, en_2, \ldots, en\}$.

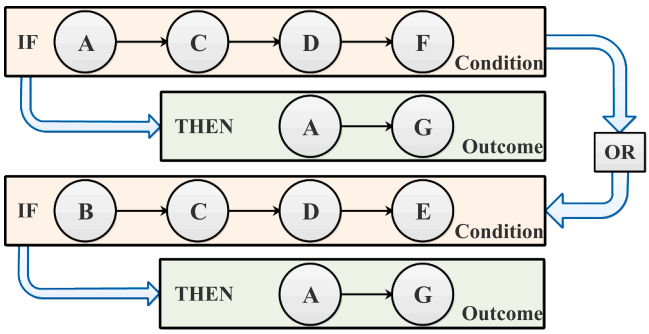

Fig. 8. Associations or logical relations between semantic paths.

4. Semi-automatic construction method of DCKG

The modeling process with which the design code representation will be utilized is crucial for successful implementation.(Macit İlal and Günaydın, 2017). To this end, we propose an accompanying modeling method for DCKG semi-automatic construction, as shown in Fig. 14. It includes four steps: interpretation, reconstruction, organization, and implementation. The method's core is the clear and well-defined semantic interpretation and knowledge reconstruction process. A design code annotation platform is also developed to assist in the DCKG construction.

4.1. Interpretation

The first stage of DCKG construction is semantic interpretation. To begin with, we follow the existing code organization to obtain the independent clause texts. The clauses texts in Chinese design codes start with a fixed format number, i.e., *"Chapter No. Section No. Clause No."*, so it can be well acquired using regular expressions. Then, the original clause text is divided by domain experts into semantic blocks, the smallest units containing independent constraint semantics for representation, combining the natural language syntax and the actual

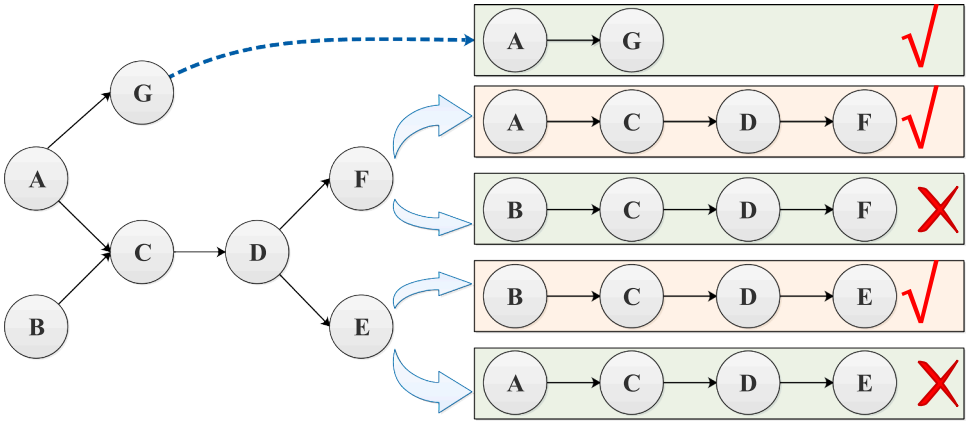

Fig. 7. All possible semantic paths contained in the clause subgraph.

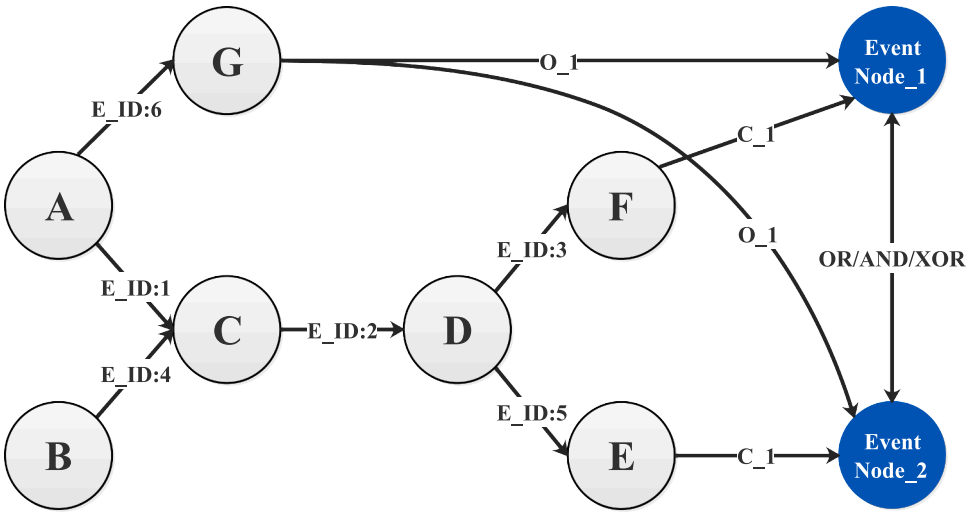

Fig. 9. Representation of event schema.

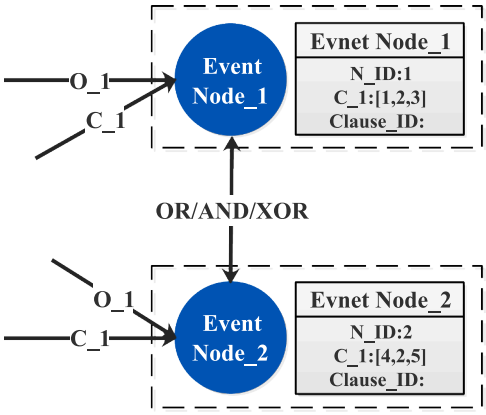

Fig. 10. Graphical representation of event node attribute.

semantic interpretation of clauses.

In the current stage, domain experts need to lead two parts; one is screening the clauses. DCKG is composed of result-oriented clauses, which are filtered by experts from the original clauses. Each result-oriented clause contains constraint semantics for different subjects, i.e., one or more constraints imposed on the component composition, properties, materials, and the relations between elements or spaces. The design result is a coupled product of the design process, but most of the design processes will not be directly reflected in the design result. So, most process-oriented clauses are not available for ACC, for example, clauses involving the selection of parameters in the design process, the choice of calculation methods and other explanatory notes for designers. But such clauses can be used in standardized model libraries and design software to assure design quality from both source and process (Cao et al., 2022).

The other one is capturing semantic blocks. Domain experts interpreted the actual semantics of original clause texts according to the semantic block structure <*Constraint subject, Constraint content*>. The developed annotation platform in Section 4.4 can assist in this process. In addition, the semantic block structure can be classified into two types according to whether the *Constraint content* contains constraint words (verbs, comparatives). The first type is semantic blocks containing constraint words. Such type is short sentences (e.g., "*height greater than 3m*", "*station equipped with elevator*"). The second type is semantic blocks without constraint words. Such type is mostly phrases or noun definite structures (e.g., "*A of B, Rated speed of escalator*" or "*AB, Stair flight tread*"). As shown in Fig. 14, four semantic blocks are divided, where *#A B#* and *#Attribute C of B#* belong to the second type, and *#B is in state E#* and *#C is greater than D#* belong to the first type.

Capturing semantic blocks also requires domain experts to explore a balance for identifying *Constraint subject*. For example, whether the "*Fire door*" should exist as a separate semantic block, i.e., the door type is fireproof. Or it should exist as an indivisible subject in a semantic block composed of short sentences. We leave this balance to expert judgment to develop a uniform processing template to ensure that the same constraint subjects are treated similarly.

4.2. Reconstruction

The second stage of DCKG construction is representation reconstruction. First, the semantic blocks are reconstructed using tuple structures. Then, the same concepts, entities and relations defined multiple times in different tuples are fused to obtain the clause subgraphs. Finally, ambiguities and redundancies in the clause subgraphs are eliminated based on the expanded schema.

Currently, the representation of semantic blocks using tuple structures also requires the interpretation and participation of domain experts to guarantee the accurate representation of actual semantics. This process involves the following three operations.

a) There are no constraint words in the second type of semantic

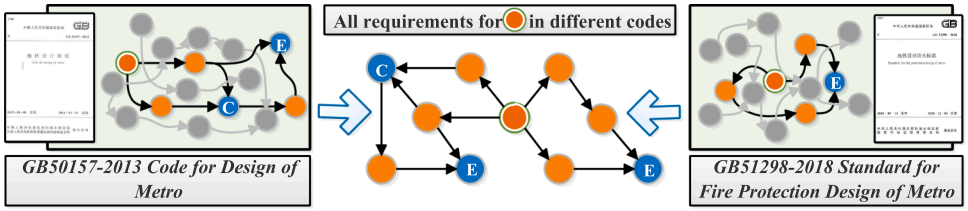

Fig. 11. All clauses of the same object in different design codes.

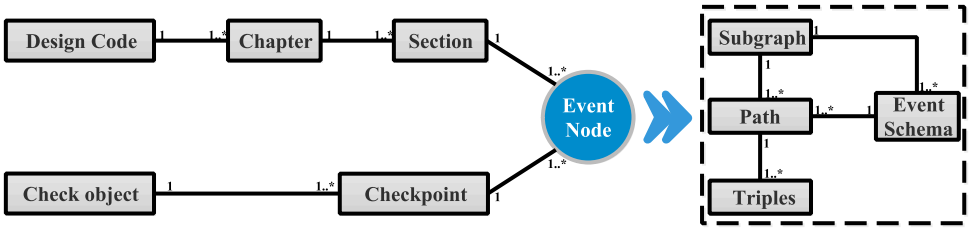

Fig. 12. Representation of integrated schema.

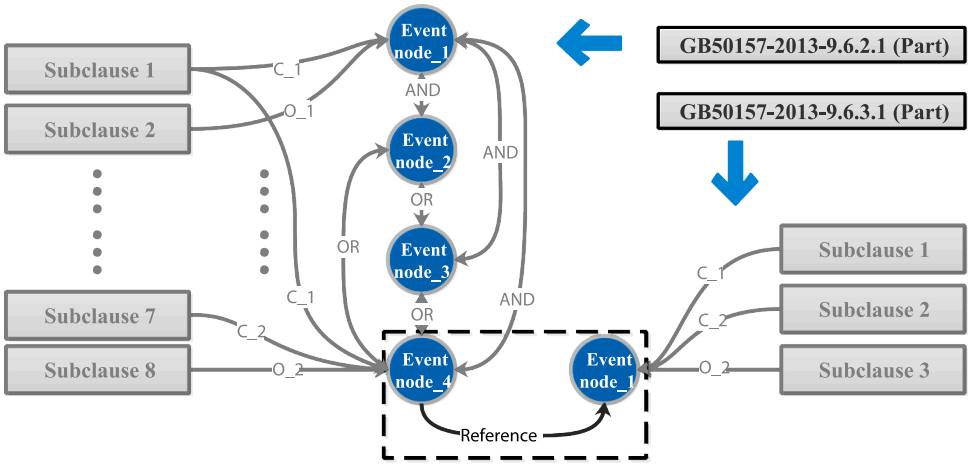

Fig. 13. Representation of reference relation.

block. The constraint semantics are hidden in the structure or ignored as common knowledge. The constraint relations need to be added explicitly to reconstruct semantic blocks as triples. Table 4 lists the constraint relations commonly used for extensions.

b) When the semantic block belongs to the first type and does not contain nested semantic. The semantic blocks reconstructed using the tuple structure are <*Entity*, *Unitary Relation*>, <*Entity_1*, *Entity_2*, *Binary Relation*>, or <*Entity_1*, *Entity_2*, *Entity_3*, *Multiple Relation*>. The relations are constraint words and the entities are *Constraint subject* and remaining *Constraint content*.

c) When the semantic block belongs to the first type and contains nested semantic. The semantic blocks reconstructed using the tuple structure are <*Entity_1*, *Entity_2*, *Entity_3*, *Relation_1*, *Relation_2*>. *Relation_2* is the constraint word of the current semantic block and the *Constraint subject* is further split into *Entity_1, Entity_2,* and *Relation_1*.

Afterward, an automatic method was designed to fuse tuples to generate clause subgraphs. And the subgraphs are visualized by ECharts on the developed annotation platform. The pseudo-code of the process is shown below.

Pseudo-code for tuple collection structure transformation

Input: Tuple, Entity, Relation, EntityDictionaries;
Output: InitialClauseSubgraph
1: Determine the RelationType of the current Tuple based on the number of Entities and Relations contained in the Tuple
2: Get the Tuples corresponding to the current clause and form the TuplesList
3: For Tuple in TuplesList:
4: If RelationType is "Unitary":
5: Add OrderNode to EntityDictionaries
6: **SetLink**(Entity_1, Relation, Entity_1);
7: **SetLink**(OrderNode, Default, Entity_1);
8: EndIf
9: If RelationType is "Binary":
10: **SetLink**(Entity_1, Relation, Entity_2);
11: EndIf
12: If RelationType is a "Multiple":
13: Create a new node ComplexNode, and

(*continued on next column*)

(*continued*)

Set ComplexNode.name = Relation.name
14: Add the ComplexNode to EntityDictionaries
15: **SetLink**(Entity_1, Part_1, ComplexNode);
16: **SetLink**(Entity_2, Part_2, ComplexNode);
17: **SetLink**(Entity_3, Part_3, ComplexNode);
18: EndIf
19: If RelationType is "Nested":
20: Create a new node ComplexNode, and
Set ComplexNode.name = Relation.name
21: Add the ComplexNode to EntityDictionaries
22: **SetLink**(Entity_1, Part_1, ComplexNode);
23: **SetLink**(Entity_2, Part_2, ComplexNode);
24: **SetLink**(ComplexNode, Relation_2, Entity_3);
25: EndIf
26: EndFor
27: Get all links to form a LinkListt
28: Wrap LinkList and EntityDictionaries in JSON
29: Initialize ECharts instance and pass in JSON data;
30: Generate InitialClauseSubgraph;
31: Return;
Function: SetLink(Source, Relation, Target)
1: Wrap Source, Relation, and Target in Link
2: Return Link

In addition, the reconstructed clause subgraphs cannot yet accurately represent the actual semantics of original clause. It requires domain experts to correct the subgraph based on the event schema mentioned in Section 3.2. First, the full path of clause subgraph is automatically obtained by random wandering. Then, the paths are filtered by domain experts according to the actual semantics of original clause. After that, new event nodes are created and directed edges connect the correct paths to complement the event schema. When the clause subgraph contains order nodes, experts are required to correct the edges pointed out from the order nodes mentioned in Section 3.1.1.

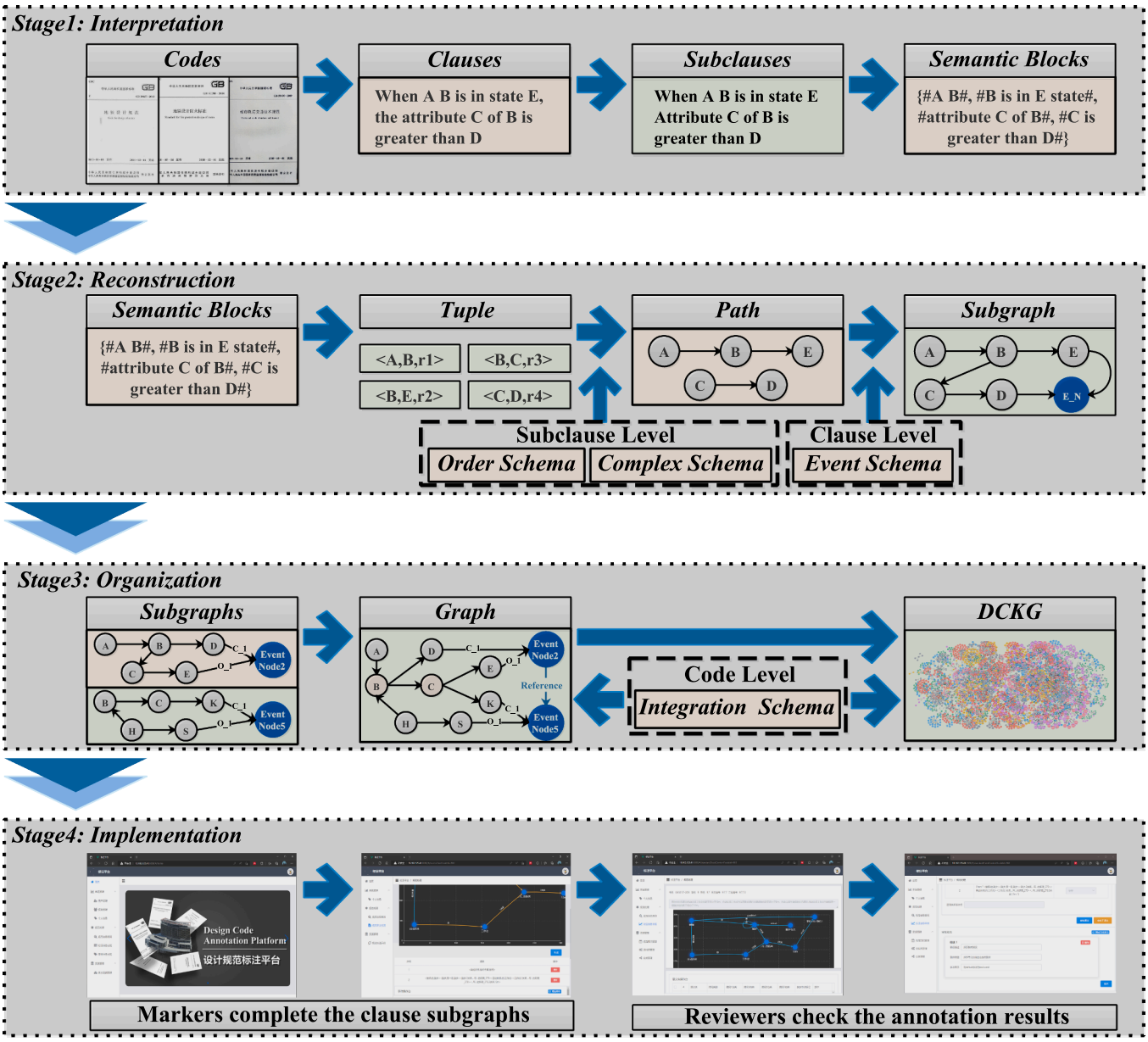

Fig. 14. Semi-automatic construction process of DCKG.

4.3. Organization

The currently obtained subgraphs can accurately and comprehensively represent the semantic information of independent clauses. The next step is to complete the higher-level semantics, i.e., the reference relation between clauses and the schema layer of DCKG.

The clause subgraph pairs with reference relation are automatically retrieved. Next, the starting event node of the reference relation is automatically located. Then, the domain expert selects the tail node among the event nodes of relevant clause subgraph. Finally, a new directed edge with *"Reference"* label is created between them.

The schema layer of DCKG has two parts. One is the *DCKG Knowledge Schema* which is used to interpret the design knowledge of DCKG. Different domain experts from different departments write the design codes. The same concepts in different design codes lack uniform interpretation and representation, and there is no clear classification

Table 4
Constraint relations commonly used for extensions.

Relations	Explanation
hasAttribute	For connecting entity objects and attribute objects.
subClassOf	For connecting entity objects that have a contextual relation.
assigns	For connecting concepts and instances that have instantiation relations.
connection	For connecting spatially connected architectural space objects.
associates	For connecting two related entity objects, one of which is an architectural element object.
aggregation	For connecting two objects with a part-whole relation, the part can exist independently of the whole.
composition	For connecting two objects that have a relation between a part and a whole, where the part cannot exist independently of the whole.

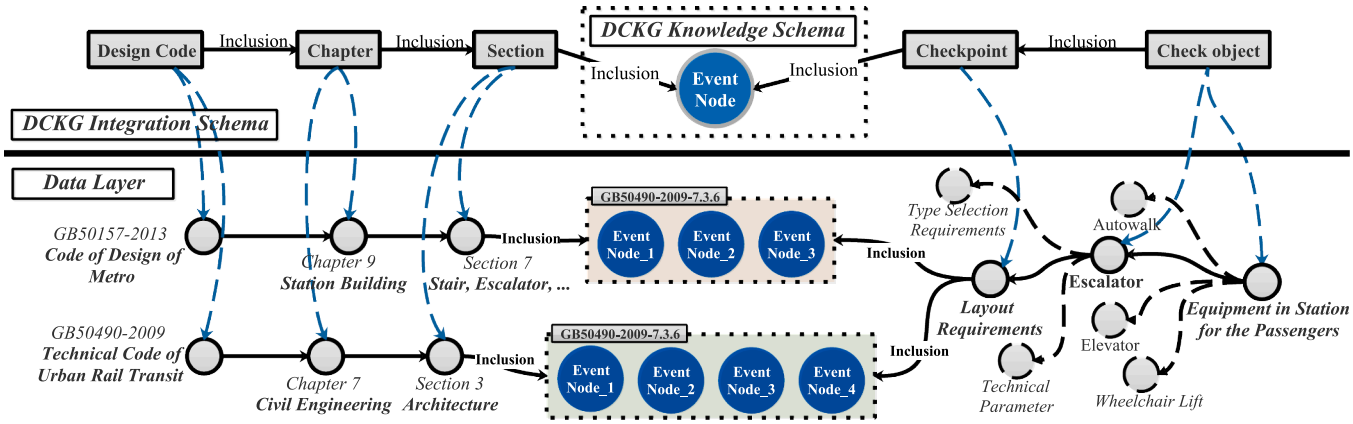

Fig. 15. Knowledge integration of DCKG.

standard for concepts. Using a bottom-up approach to construct the DCKG knowledge schema is more feasible. First, we completed the design code representation and DCKG construction. Then the experts combine domain knowledge to gradually abstract the upper-level concepts and hierarchies from the entities and relationships in DCKG by clustering and classification operations.

The other part is the *DCKG Integration Schema*, which is used to manage the events of DCKG. The hierarchical organization of codes is constructed automatically. The hierarchy graph consisting of "*Code-Chapter-Section*" is automatically built using the catalog in design code. The edge "*Inclusive*" between the node "*Section*" in the hierarchy graph and the event node is created according to the value of the event node's property "*Clause_ID*". The left half of Fig. 15 shows the hierarchical organization of two clauses.

Then, the design code is organized in terms of events, combined with *Check objects* and *Checkpoints*. Among them, The *Check objects* are determined based on the concepts in the *Code Knowledge Schema* and *GB/T51269–2017 Standard for classification and coding of building information model*. And the *Check objects* mainly include the component elements divided by function or space and the design parts with interrelations. The *Checkpoints* are determined by combining the clauses involved in the *Check objects* and *GB/T51301–2018 Standard for design delivery of building information modeling*. And the *Checkpoints* are associated with the corresponding event nodes in DCKG through edges with "*Inclusive*" label. Note that a *Checkpoint* can contain multiple events, and different events of the same clause may belong to different *Checkpoints*.

An example is given in Fig. 15, where the right half shows the specific integration schema. The *Check object* "*Equipment in station for the passenger*" includes four other *Check objects*: "*Autowalk, Escalator, Elevator, and Wheelchair Lift*". The *Check object* "*Escalator*" contains three *Checkpoints*: "*Layout requirements, Selection requirements and Technical parameters*". Among them, the *Checkpoint "Layout requirements"* involve seven events in clauses *GB50157–2013–9.7.7* and *GB50490–2009–7.3.6*.

4.4. Implementation

A design code annotation platform is developed to assist domain experts in implementing the construction of high-quality DCKG. We have publicly released a small-scale DCKG built with this platform, available for download at (https://github.com/Yang-Mingsong/DCKG4ACC). The platform is developed based on the spring boot framework and visualized by ECharts to present clause subgraphs. Afterward, Neo4j is used to store the DCKG knowledge data. The roles of the platform include administrator, marker, and reviewer. Fig. 16 shows the use case diagrams.

The administrator manages the rights to the functions of all users and is responsible for marking task assignments, review task assignments, and design code data updates. The administrator also has access to the information query function.

The marker receives marking tasks assigned by the administrator and is responsible for marking semantic blocks, reconstructing tuples, selecting semantic paths, and marking events and relations. The marked-up code clauses will be submitted to the administrator pending the assignment of review task. The marker also has the function rights of information search and review comment viewing to facilitate the modification of the original marking.

The reviewer receives review task assigned by the administrator, and determines whether the marked results are accurate. The approved clauses marking data will be updated in DCKG. When the review does not pass, the reviewer will submit a review comment to the marker. The reviewer also has the function rights to update and query the corpus.

The platform adds a clause subgraph review mechanism to ensure the quality of DCKG. Specifically, the submitted annotation data is inspected by reviewers in three aspects, a) whether the semantic blocks are complete, b) whether the annotation of similar entities and relations are consistent, and c) whether the event schemas are accurately defined. The annotated data that pass the review will be updated into the corpus and DCKG.

5. Experiment and case study

5.1. Comparison experiment

GB50157–2013 Code for design of metro, as a Chinese national code, is one of the core codes that the design of metro project should comply with. We adopt all the clauses of Chapter 9 "*Station Building*" in *GB50157–2013* for the experiment. The selected clauses are representative of the Chinese codes, and the distribution of clauses in each section is shown in Table 5. We are dividing the clauses into process-oriented and result-oriented, as mentioned in 4.1. Fifty-nine clauses (67 %) are result-oriented clauses available for ACC, and twenty-nine (33 %) are process-oriented clauses.

Further analysis found 31 multiple event clauses in the code clauses available for ACC, as shown in Table 6. The single event clauses are divided into two categories, one is composed of multiple subclauses, and there are 20 clauses in this category. The second is composed of single subclause. For example, *GB50157–2013–9.8.7 Barrier-free toilets should be designed in the stations*, and there are eight clauses in this category. In contrast, complex clauses with multiple requirements are more common in design codes.

The count of clauses including different semantic types is also listed in Table 6, where the various semantic types are counted independently, i.e., clauses containing multiple semantic types are counted repeatedly in different columns. The analysis reveals that each clause contains binary semantic, and the semantic type present in complex clauses are

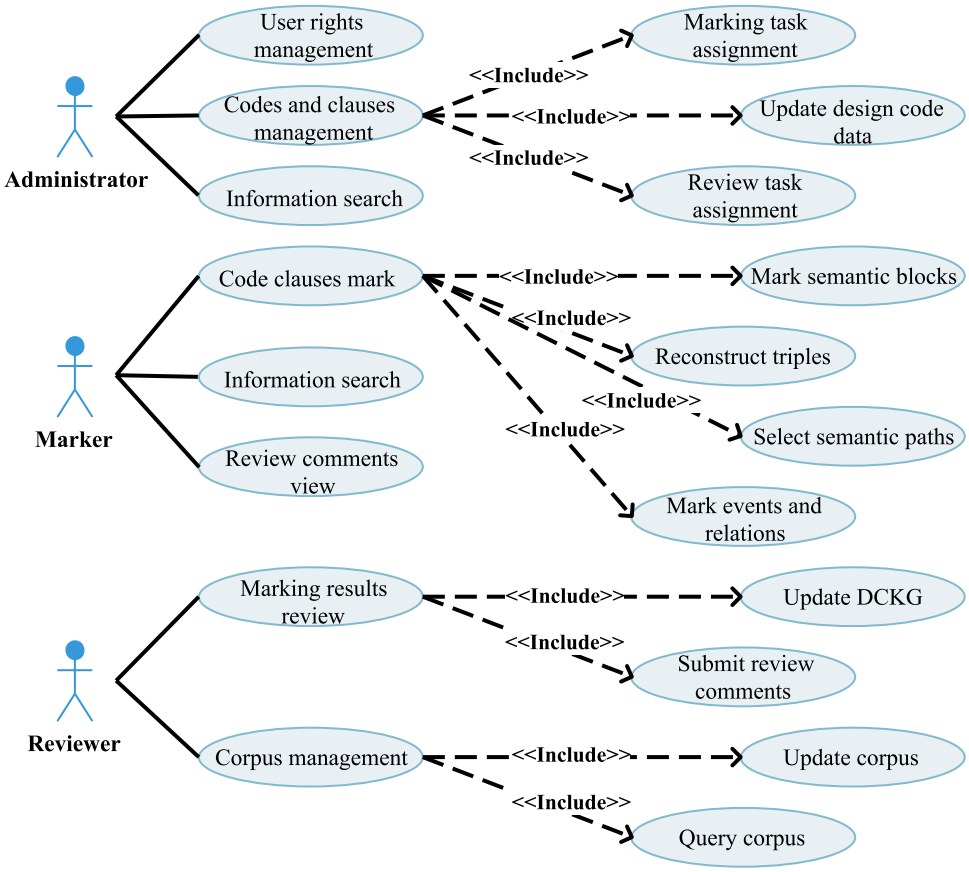

Fig. 16. Use case diagram of the administrator, marker and reviewer.

Table 5
Classification statistics of experimental clauses.

No.	Section	Total number of clauses	result-oriented clause	process-oriented clause
9.1	General Requirement	7	1	6
9.2	General Layout of the Station	6	2	4
9.3	Station Plane	17	12	5
9.4	Station Environment Design	7	4	3
9.5	Station Entrances and Exits	6	4	2
9.6	Ventilation Shaft and Cooling Tower	12	9	3
9.7	Stair, Escalator, Elevator and Platform Screen Door	13	13	0
9.8	Station Barrier-free Facilities	7	7	0
9.9	Transfer Station	5	1	4
9.10	Economize Energy of Building	8	6	2
Total	-	88	59	29

Table 6
Semantic types in different clause categories.

Clause category	Count of clauses	Count of clauses including different semantic types						
		unitary semantic (Ignoring context)	unitary semantic (Considering context)	binary semantic	multiple semantic	nested semantic	reference relation	Total
Single event clause (single subclause)	8	1	0	8	0	0	0	9
Single event clause (multiple subclauses)	20	2	5	20	0	6	3	36
Multiple events clause	31	1	20	31	4	18	5	79

more diverse. There are both unitary semantics (considering context) and nested semantics in more than half of the clauses which contain multiple events.

The model that can represent the most semantic types in each category of Table 1 was selected for comparison with our model. Results of the comparison experiment are shown in Table 7, where our model outperforms other models in terms of representation ability. It can represent a wider range of semantic types and a greater number of clauses. The detailed analysis data for the experiment are published at (https://github.com/Yang-Mingsong/DCKG4ACC). Unlike Table 1, the columns "*Numerical, Non-numeric*" are combined because all models can support binary semantics. And to accurately elaborate on the complexity

Table 7
Comparison table of representation among seven models.

Model	Literature	Count of semantic types and clauses	Subclause level					Clause level			Code level
			Unitary semantic		Binary semantic	Multiple semantic	Nested semantic	Relations between subclauses		Relations between subclause sets	Reference relation between clauses
			Ignoring context	Considering context				Single subclause	Multi-subclauses		
Commercial software model	(Solibri, 2021)	4 √ 17	√ 3	× 0	√ 17	× 0	× 0	√ 8	√ 9	× 0	× 0
Logic-based model	(Nawari, 2019)	7 √ 28	√ 4	× 0	√ 28	√ 2	√ 6	√ 8	√ 12	√ 8	× 0
Object-oriented model	(Malsane et al., 2015)	5 √ 20	√ 3	× 0	√ 20	× 0	√ 3	√ 8	√ 12	× 0	× 0
Semantic web-based model	(Jiang et al., 2022)	6 √ 26	√ 4	× 0	√ 26	× 0	√ 6	√ 8	√ 12	√ 6	× 0
New modeling languages	(Macit İlal and Günaydın, 2017)	6 √ 24	√ 4	× 0	√ 24	× 0	× 0	√ 8	√ 12	√ 4	√ 4
Visual language	(Kim et al., 2019)	5 √ 20	√ 4	× 0	√ 20	× 0	× 0	√ 8	√ 9	√ 3	× 0
Our model		**9 √** **59**	**√** **4**	**√** **25**	**√** **59**	**√** **4**	**√** **24**	**√** **8**	**√** **20**	**√** **31**	**√** **8**

of different clauses, we subdivide the column "*Relations between subclauses*" into two columns "*Single subclause, Multiple subclauses*". The column "*Hierarchical organization of codes*", which does not involve the internal semantics of clauses, is deleted.

The sum of the three columns "*Single subclause*, *Multi-subclauses*, and *Relations between subclause sets*" at the clause level is the count of clauses that the current model can represent. It is also equal to the count of clauses containing binary semantics. It can be found that our model can represent all semantic types and clauses.

The analysis revealed that all models could accurately represent the single event clauses composed of single subclause since such clauses usually contain only binary semantic relations or unitary semantic (ignoring context).

For the single event clauses composed of multiple subclauses, our model can represent more clauses than the other models because there are already a few complex semantics in this clause category. Table 6 shows that 5/20 clauses include unitary semantic (considering context), and 6/20 clauses include nested semantic.

For the multiple events clauses, four models can represent the semantic and logical relations between events, i.e., relations between subclause sets, in addition to our model. However, the ability to represent complex semantics limit the applicability of these four models compared to ours. Table 6 shows 20/31 clauses including unitary semantic (considering context), 4/31 clauses including multiple semantic, and 18/31 clauses including nested semantic. These four models can only represent up to 28/59 clauses, which is less than 50 %. Our model purposefully proposes order, complex and event schemas to support the representation of such semantic types. At the same time, our model also supports the reference relations at the code level through the proposed integration schema.

5.2. Case study

In the case study, some typical clauses in the *GB50157–2013 Code for design of metro* are represented by our proposed schemas as examples. After that, taking the IFC model of metro station as an example, the correctness and feasibility of DCKG representation model are verified for checking by selecting a presentative code clause.

5.2.1. Implementation of order schema

Clause 1: *GB50157–2013–28.6.1 The final distribution box for fire-fighting electrical equipment should be equipped with automatic switching devices.*

Clause 2: *GB50157–2013–9.6.3.4 The bottom of the air shaft should set up drainage facilities.*

Fig. 17 illustrates the subgraphs of the above clauses. The newly added order node in Fig. 17a) points to the node "*Distribution box*", and the label "*Inward*" is chosen for the edge to represent the semantic order accurately, equivalent to *associate*(*Fire-fighting Electrical Equipment*, *final*(*Distribu-tion Box*))∩*equip*(*final*(*Distribution Box*), *Automatic Switching Device*). In Fig. 17b), the order node pointing to the node "*Air shaft*", the edge label selected "*Backward*", and the semantic of the current clause is equivalent to *setUp*(*Bottom*(*Air Shaft*), *Drainage Facilities*).

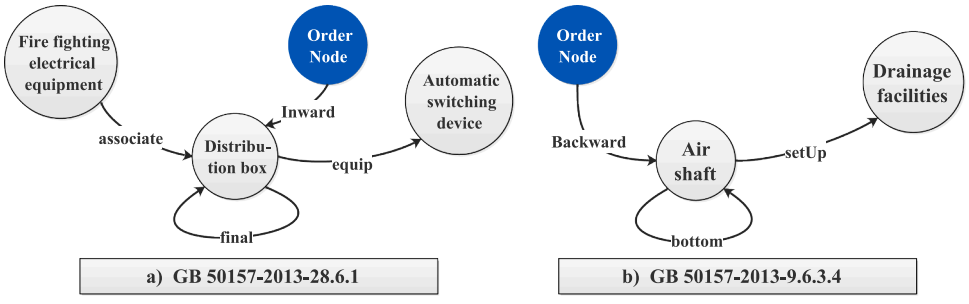

Fig. 17. Examples of the order schema in the code clauses.

5.2.2. Implementation of complex schema

Clause 1: *GB50157–2013–28.4.22.2 The junction of the horizontal trunk pipe and the vertical main pipe of the ventilation and air conditioning system should be set up as the fire damper.*

Clause 2: *GB50157–2013–9.3.7. The distance between the ticket gate and the stairs should be greater than or equal to 5m.*

Clause 3: *GB50157–2013–13.4.1.2. The ventilation system should ensure that the interval tunnel meets the effective ventilation.*

The complex schemas in Fig. 18a) and b) represent nested semantics. The complex node "*Junction of.and.*" represents exactly the spatial location where the fire dampers should be set, and the complex node "*Distance between.and.*" accurately represents the specific subject constrained by greater than or equal to 5m. Fig. 18c) shows a clause subgraph containing multiple semantic. Through complex node ". *Ensure that.meets.*" and three directed edges, the multiple semantic between "*Ventilation system*", "*Zone tunnel*" and "*Effective ventilation*" are accurately represented.

5.2.3. Implementation of event schema

Clause 1: *GB50157–2013–25.1.15 When the rated speed of the escalator is 0.5m/s, and the lifting height is not greater than 6m, the number of upper and lower horizontal steps shall not be less than 2; when the rated speed is equal to 0.65m/s, the number of upper and lower horizontal steps shall not be less than 3; when the rated speed is greater than 0.65m/s, the number of upper and lower horizontal steps shall not be less than 4.*

The clause constrains the number of horizontal steps for escalators in different conditions. Fig. 19 illustrates the clause subgraph. The event node and directed edges between it and the tail node of corresponding subclause paths are added to represent implicit association relations between subclause paths. The bidirectional edges *"OR"* are added to represent logical relations between events.

Fig. 20 shows the event_1 in the above subgraph, which is equivalent to:

IF C_1: (<*Escalator*, *hasAttribute*, *Ratedspeed*>, <*Ratedspeed*, =, *0.5m/s*>) *and C_2*: (<*Escalator*, *hasAttribute*, *LiftingHeight*>, <*LiftingHeight*, <=, *6m*>).

THEN O_2: (<*Escalator*, *composition*, *HorizontalStep*>, <*HorizontalStep*, *hasAttribute*, *Quantity*>, <*Quantity*, >=, 2>).

Fig. 21 shows the event_2 in the above subgraph, which is equivalent to:

IF C_1: (<*Escalator*, *hasAttribute*, *Ratedspeed*>, <*Ratedspeed*, =, *0.65m/s*>).

THEN O_1: (<*Escalator*, *composition*, *HorizontalStep*>, <*HorizontalStep*, *hasAttribute*, *Quantity*>, <*Quantity*, >=, 3>).

Fig. 22 shows the event_3 in the above subgraph, which is equivalent to:

IF C_1: (<*Escalator*, *hasAttribute*, *Ratedspeed*>, <*Ratedspeed*, >, *0.65m/s*>).

THEN O_1: (<*Escalator*, *composition*, *HorizontalStep*>, <*HorizontalStep*, *hasAttribute*, *Quantity*>, <*Quantity*, >=, *4*>).

5.2.4. Model checking

(1) Clause subgraph and IFC model preparation

The IFC model to be checked is shown in Fig. 23 using the BIMVision 2.25.3. The IFC standard using the EXPRESS language defines entities and relationships. The IFC-SPF file expresses and stores the model data through numerous instances of entities and relationships.

Fig. 24 shows the selected clause subgraph, which contains the order, complex and event schemas, and involves multiple types of mapping and checking. And Fig. 15 shows that this event belongs to the *Checkpoint* "*Layout requirements*" of the *Check object* "*Escalator*".

Original clause: *GB50157–2013–9.7.7 …When the escalator is opposite the stair, the distance between the working point of the escalator and the first tread of the stair shall not be less than 12m.*

First, we get the "*C_1*" and "*O_1*" paths of the current event according to the property of the event node, as shown in Fig. 25. Then get the triple set of "*C_1*" path: ***T_Set***={<*Escalator*, *opposite*, *Stair*>}, and combine the contained order schema and complex schema to obtain the triple set of "*O_1*" path: ***T_Set***={<*Escalator*, *hasAttribute*, *Working point*>, <*Stair*, *aggregation*, *Tread*>, <*Tread*, *first*, *Tread*>, <*Tread*, *P_1*, *C_N*>, <*Working point*, *P_2*, *C_N*>, <*C_N*, ≥, *12m*>}, where *C_N* is the complex node "*Distance between.and.*".

(2) Checking and results

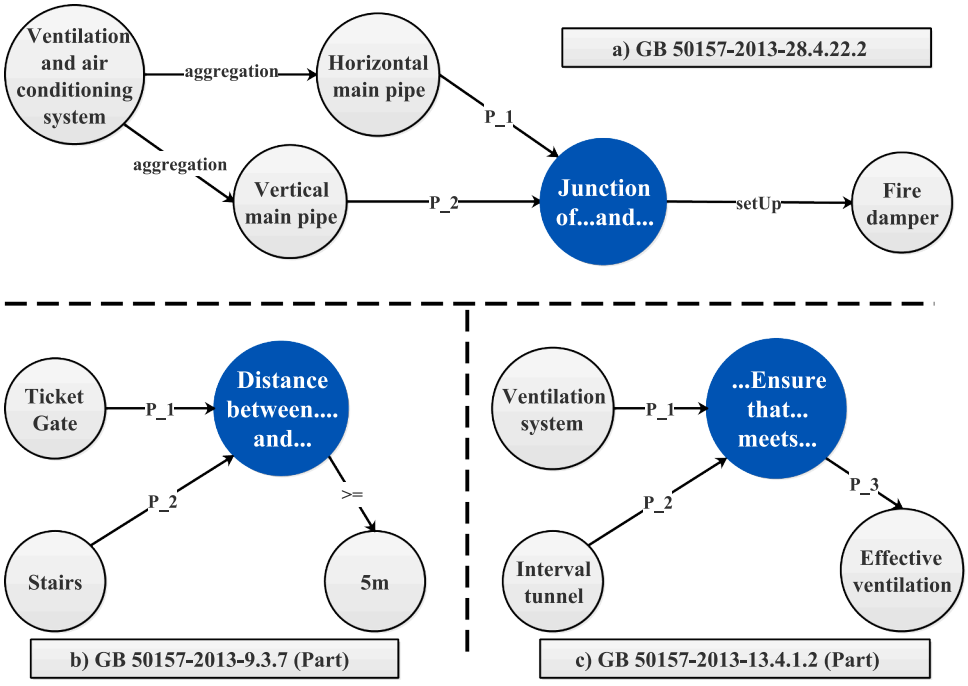

Fig. 18. Examples of the complex schema in the code clauses.

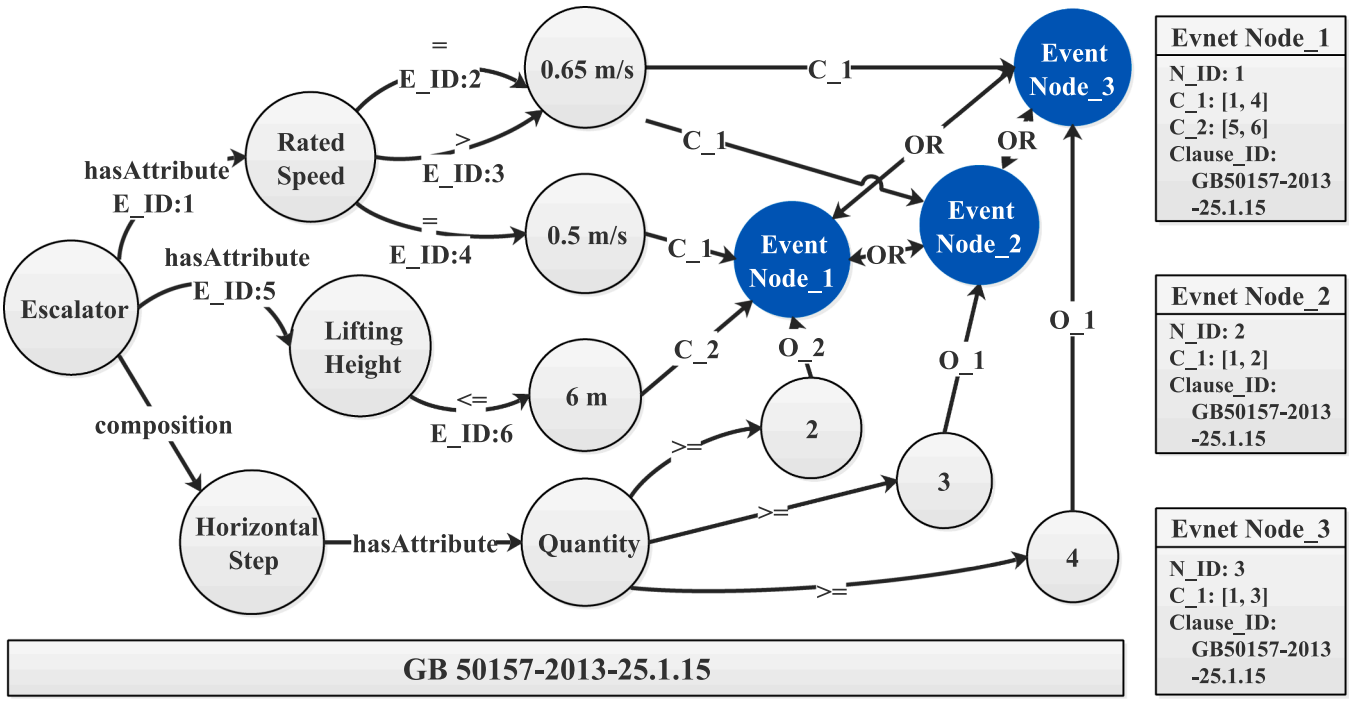

Fig. 19. Example of the event schema in the clause subgraph.

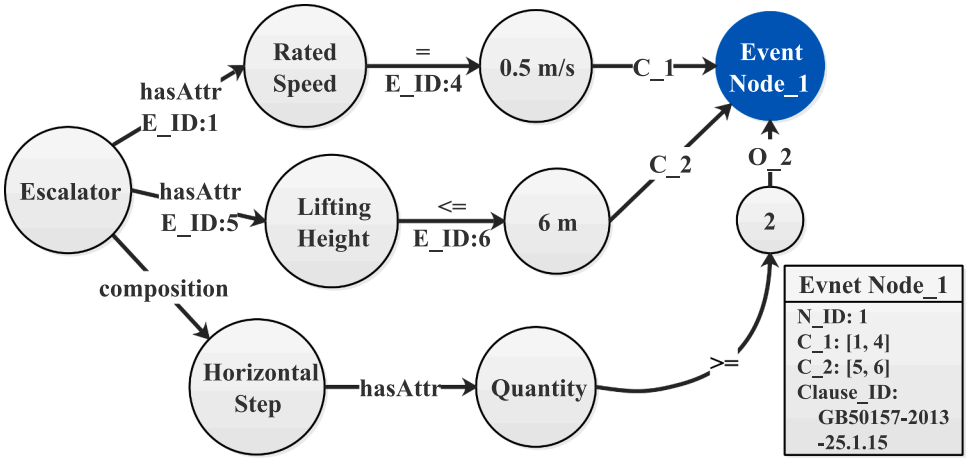

Fig. 20. Part of event_1.

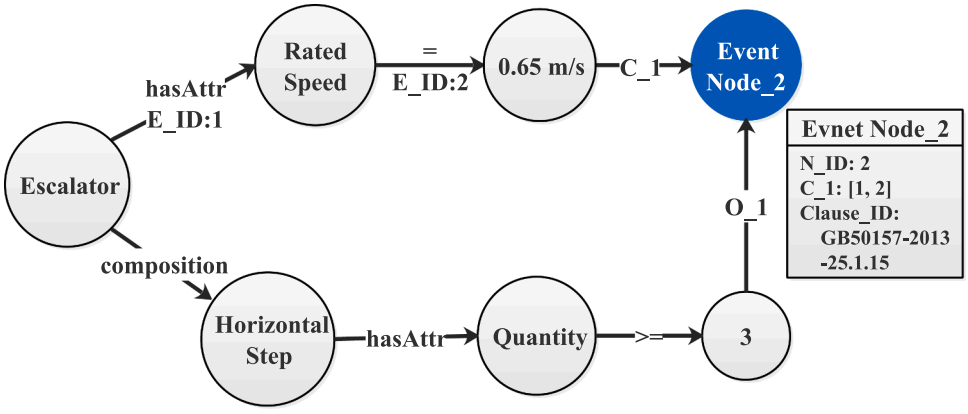

Fig. 21. Part of event_2.

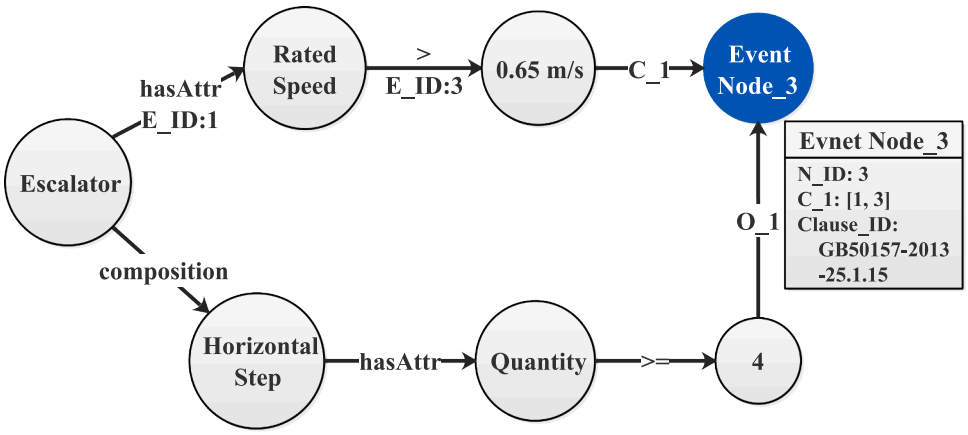

Fig. 22. Part of event_3.

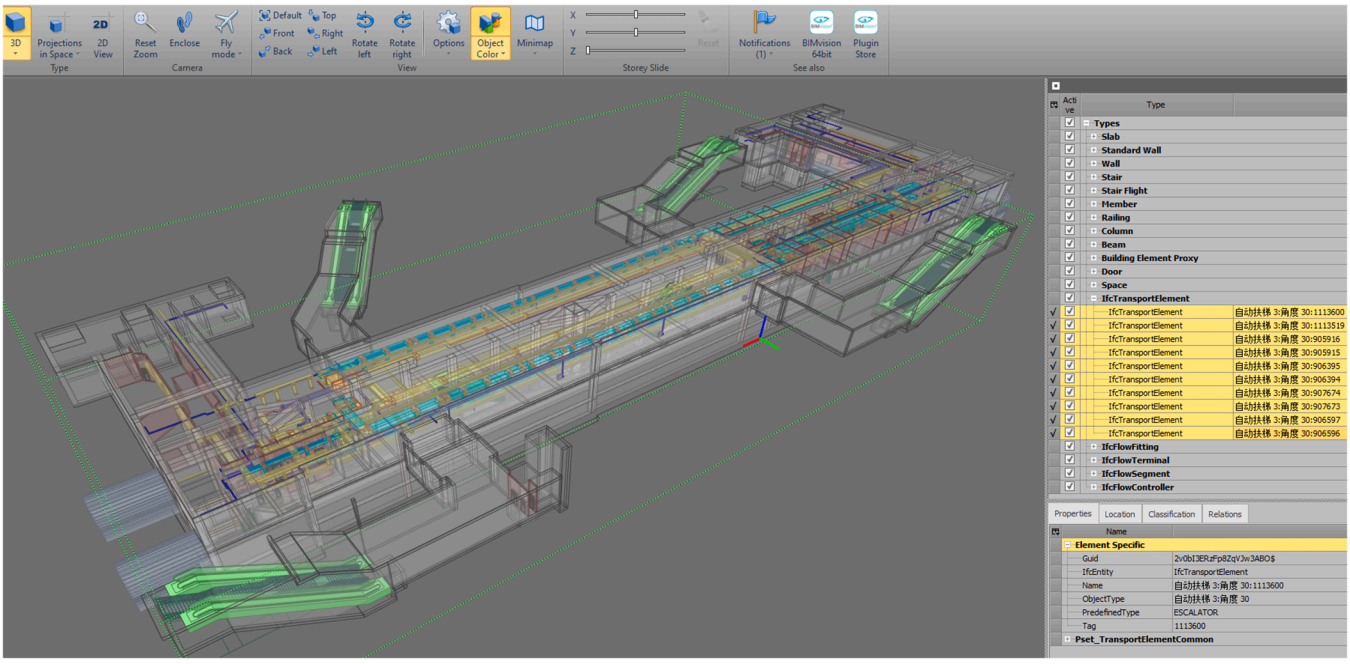

Fig. 23. The metro station model for compliance checking.

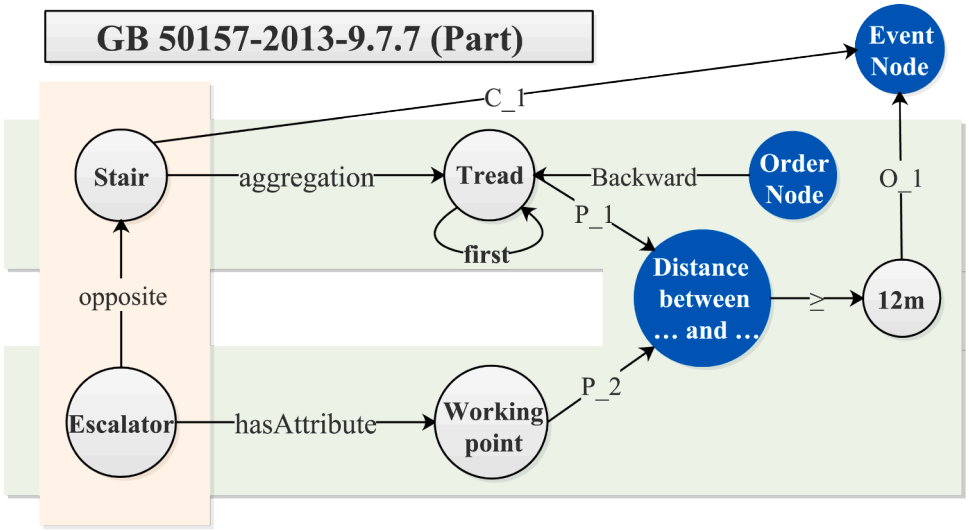

Fig. 24. Clause subgraph used for checking.

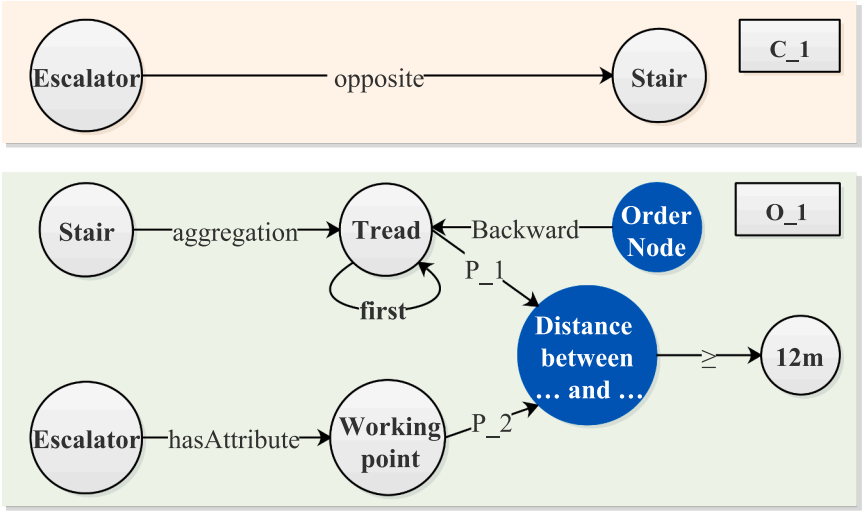

Fig. 25. *C_1* path and *O_1* path of current event.

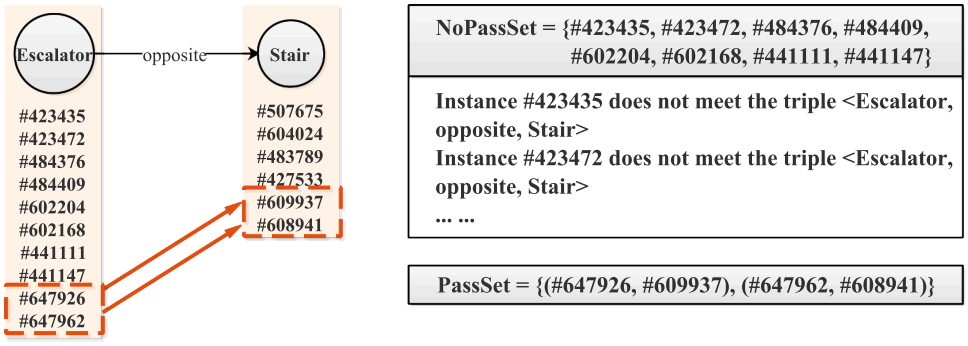

Fig. 26. Checking process and results of *C_1* path.

Table 8
Checking process and results of the triple <*Escalator*, *opposite*, *Stair*>.

Instance set	Instances and combinations									
Escalator_set	#423435	#423472	#484376	#484409	#602204	#602168	#441111	#441147	#647926	#647962
	#e1	#e2	#e3	#e4	#e5	#e6	#e7	#e8	#e9	#e10
Stair_set	#507675	#604024	#483789	#427533	#609937	#608941				
	#s1	#s2	#s3	#s4	#s5	#s6				
Unpassed_set	#e1	#e2	#e3	#e4	#e5	#e6	#e7	#e8		
Passed_set	(#e9, #s5)	(#e10, #s6)								

The triples are obtained sequentially from the ***T_Set*** of paths, and then the instances or combinations in the IFC-SPF file corresponding to the triple nodes are queried through the predefined mapping patterns. Afterward, the checking is executed according to the semantic information embedded in the edge of triples. The checking results are recorded in two newly created sets, ***Passed_Set*** and ***Unpassed_Set***. Note that the process data of checking within the same event will be shared for subsequent checking, and the logical relation between paths will be "*and*" by default.

a) Checking of "C_1" path:

Fig. 26 illustrates the checking process and results of the "*C_1*" path. This path contains only one triple <*Escalator*, *opposite*, *Stair*>, where the *Escalator* in triple cannot be mapped to a separate entity in IFC Schema. But it can be mapped to the entity *IfcTransportElement* whose property *PredefinedType* takes the value ". *ESCALATOR.*" The set of escalator instances can be obtained by the above mapping pattern, i.e., ***Escalator_Set***={*#423435*, *#423472*, *#484376*, *#484409*, *#602204*, *#602168*, *#441111*, *#441147*, *#647926*, *#647962*}. For the sake of brevity, we simplify it to ***Escalator_Set***={*#e1*, *#e2*, *#e3*, *#e4*, *#e5*, *#e6*, *#e7*, *#e8*, *#e9*, *#e10*}. Get the set of stair instances in the same way, i.e., ***Stair_Set***={*#s1*, *#s2*, *#s3*, *#s4*, *#s5*, *#s6*}. The edge "*opposite*" corresponds to the predefined geometric calculation method.

As shown in Table 8, where the instance combinations (*#e9*, *#s5*) and (*#e10*, *#s6*) that satisfy the edge "*opposite*" were found.

a) Checking of "*O_1*" path:

Fig. 27 illustrates the checking process and results of the "*O_1*" path. Six triples contained in this path are checked in turn. The instances of *Escalator* in triple <*Escalator*, *hasAttribute*, *Working point*> take the checking result of "*C_1*" path, i.e., ***Escalator_Set***={*#e9*, *#e10*}. However, the property set *Pset_TransportElementCommon* of element *IfcTransportElement* does not contain property "*Working point*". We added the "*Working point*" as the *IfcPropertySingleValue* entity using the custom

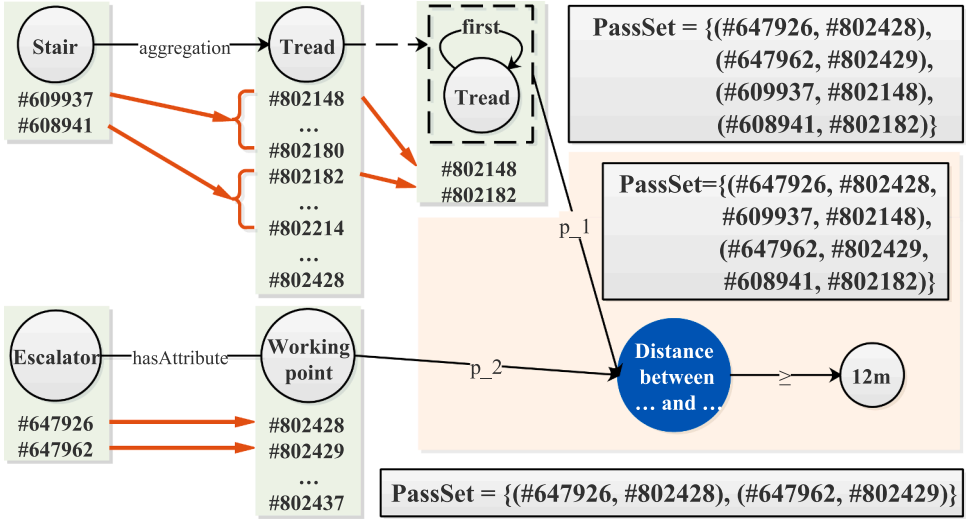

Fig. 27. Checking process and results of the *O_1* path.

Table 9
Checking process and results of the triple <*Escalator*, *hasAttribute*, *Working point*>.

Instance set	Instances and combinations									
Escalator_set	#647926 #e9	#647962 #e10								
Working point_set	#802428 #w1	#802429 #w2	#802430 #w3	#802431 #w4	#802432 #w5	#802433 #w6	#802434 #w7	#802435 #w8	#802436 #w9	#802437 #w10
Unpassed_set										
Passed_set	(#e9, #w1)	(#e10, #w2)								

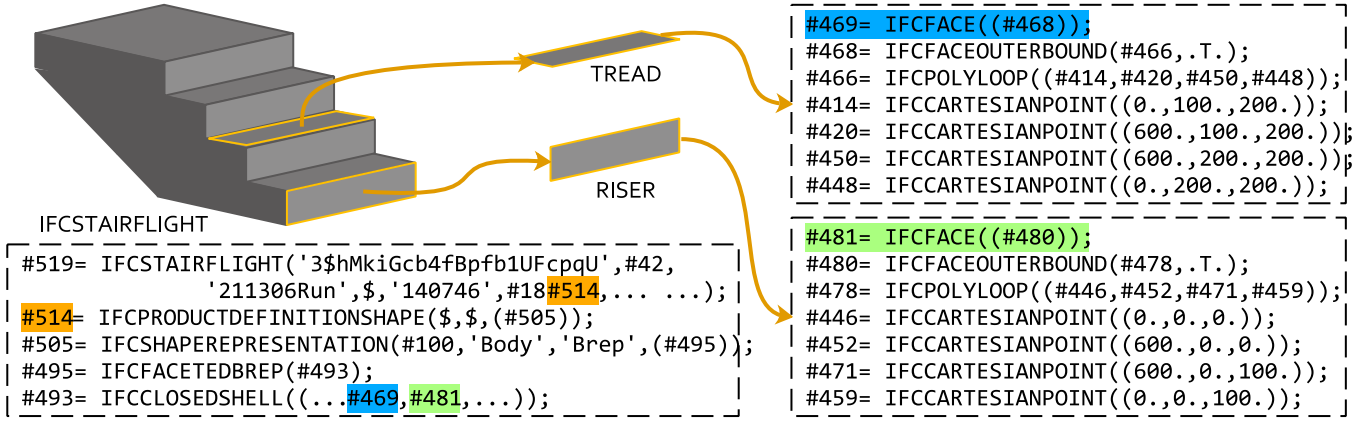

Fig. 28. Geometric information of the tread and riser in the *IfcStairFlight* instance.

property set provided by IFC Schema. And then, we get the ***WorkingPoint_Set***={*#w1*, *#w2*, ., *#w10*}. The edge "*hasAttribute*" corresponds to a query template "*?Subject*(*IfcTransportElement*) → *IfcRelDefinesByProperties* → *IfcPropertySet* → *?Object*(*IfcPropertySingleValue.name*='*Working point*')".

After checking by query, we get the ***Passed_Set***={(*#e9*, *#w1*), (*#e10*, *#w2*)} and ***Unpassed_Set***=*ϕ*, as shown in Table 9.

The instances of *Stair* in triple <*Stair*, *aggregation*, *Tread*> take the checking result of "*C_1*" path, i.e., ***Stair_Set***={*#s5, #s6*}. As shown in Fig. 28, the geometric representation of *Tread* can be found in IFC file. Still, there is no corresponding entity since only a limited number of concepts can be formally defined within IFC Schema. However, the proxy mechanism provides an effective extension method for undefined concepts in IFC Schema. We create new instances of *IfcBuildingElementProxy* and assign values to attributes after setting the type to "*TREAD*". Among them, the location information of *Tread* instances followed the values of *IfcStairFlight.ObjectPlacement*, and the shape information are represented by the corresponding *IfcFace* instances in *IfcStairFlight*. And then, we get the ***Tread_Set***={*#t1*, *#t2*, …, *#t256*}. The edge "*aggregation*" corresponds to a query template "*?Subject*(*IfcStair*) → *IfcRelAggregates* → *?Object*(*IfcBuildingElementProxy.PredefinedType*=*USERDEFINED* & *IfcBuildingElementProxy.ObjectType*=*Tread*)".

After checking by query, we get the ***Passed_Set***={(*#s5*, *#t1*), (*#s5*, *#t2*), …, (*#s5*, *#t32*), (*#s6*, *#t33*), (*#s6*, *#t34*), …, (*#s6*, *#t65*)} and ***Unpassed_Set***=*ϕ*, as shown in Table 10.

The instances of *Tread* in triple <*Tread*, *first*, *Tread*> take the checking result of previous triple, i.e., ***Tread_Set***={*#t1*, …, *#t65*}. Edge "*first*" corresponds to the predefined judgment method. The *Tread* instance with the smallest z-coordinate value is returned. As shown in Table 11, the ***Passed_Set***={(*#s5*, *#t1*), (*#s6*, *#t33*)} and ***Unpassed_Set***={(*#s5*, *#t2*),

Table10
Checking process and results of the triple <*Stair*, *aggregation*, *Tread*>.

Instance set	Instances and combinations									
Stair_set	#609937	#608941								
	#s5	#s6								
Tread_set	#802148	#802149	…	#802180	#802182	#802183	…	#802214	…	#802428
	#t1	#t2	…	#t32	#t33	#t34	…	#t65	…	#t256
Unpassed_set										
Passed_set	(#s5, #t1)	(#s5, #t2)	…	(#s5, #t32)	(#s6, #t33)	(#s6, #t34)	…	(#s6, #t65)		

Table11
Checking process and results of the triple <*Tread*, *first*, *Tread*>.

Instance set	Instances and combinations							
Tread_set	#802148	#802149	…	#802180	#802182	#802183	…	#802214
	#t1	#t2	…	#t32	#t33	#t34	…	#t65
Unpassed_set	(#s5, #t2)	(#s5, #t3)	…	(#s5, #t32)	(#s6, #t34)	(#s6, #t35)	…	(#s6, #t65)
Passed_set	(#s5, #t1)	(#s6, #t33)						

Table 12
Checking process and results of complex schema triples.

Instance set	Instances and combinations			
Tread_set	#802148	#802182		
	(#s5, #t1)	(#s6, #t33)		
Working point_set	#802428	#802429		
	(#e9, #w1)	(#e10, #w2)		
Distance_set	(#s5, #t1, #e9, #w1)	(#s5, #t1, #e10, #w2)	(#s6, #t33, #e9, #w1)	(#s6, #t33, #e10, #w2)
	29.002	6.73	6.73	29.002
Unpassed_set	(#s5, #t1, #e10, #w2)	(#s6, #t33, #e9, #w1)		
Passed_set	(#s5, #t1, #e9, #w1)	(#s6, #t33, #e10, #w2)		

…, (*#s5*, *#t32*), (*#s6*, *#t34*), …, (*#s6*, *#t65*)}.

The triples <*Tread*, *p_1*, *C_N*>, <*Working point*, *p_2*, *C_N*> and <*C_N*, ≥, *12m*> represent the nested semantic by complex schema. The checking process and results of the current complex schema are shown in Table 12. First, obtain the instances of *Tread* and *Working point* according to the checking result of previous triples, i.e., ***Tread_Set***={(*#s5*, *#t1*), (*#s6*, *#t33*)} and ***WorkingPoint_Set***={(*#e9*, *#w1*), (*#e10*, *#w2*)}. Then, we get the instance combinations for complex node, i.e., ***Distance_Set***={(*#s3*, *#t1*, *#e9*, *#w1*), (*#s3*, *#t1*, *#e10*, *#w2*), (*#s4*, *#t33*, *#e9*, *#w1*), (*#s4*, *#t33*, *#e10*, *#w2*)}, and the complex node corresponds to the spatial distance calculation. The predefined calculation function is called to obtain the results, which are {*29.002*, *6.73*, *6.73*, *29.002*}. Finally, the checking result {*T*, *F*, *F*, *T*} is obtained by reasoning according to the edge ">". Afterward, we added the instance combination with the result "*T*" to ***Passed_Set*** and the instance combination with the result "*F*" to ***Unpassed_Set***.

After completing the checking for paths "*C_1*" and "*O_1*", we merge the checking results and remove the checking results of the "*O_1*" path that do not satisfy the results of "*C_1*" path "(*#e9*, *#s5*) *and* (*#e10*, *#s6*)". As shown in Fig. 29, we get the finally ***Passed_Set***={(*#s5*, *#t1*, *#e9*, *#w1*), (*#s6*, *#t33*, *#e10*, *#w2*)}, and feedback the non-compliant instance combinations individually from the ***Unpassed_Set***.

6. Discussion

The research presented in this paper aims to develop a design code

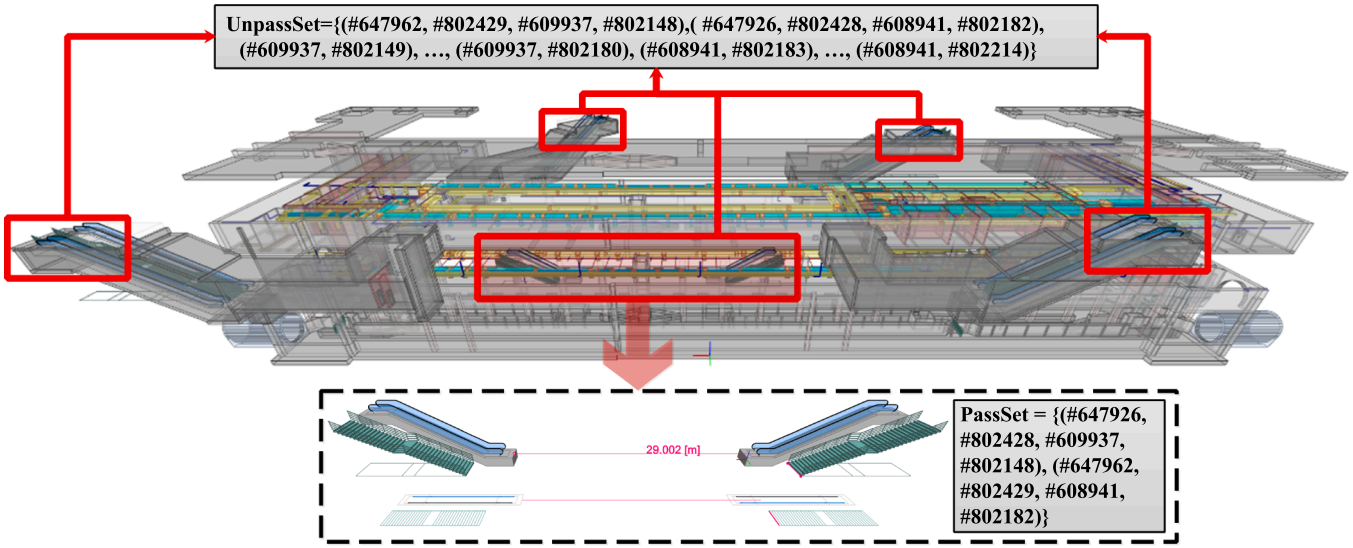

Fig. 29. Presentation of the final checking results on the IFC model.

representation model based on KG for ACC, and propose an accompanying semi-automatic method for DCKG construction. The existing representation models ignore the diverse semantic types in complex clauses of design codes. Compared with them, the significant features of our model and method are summarized as follows.

In contrast to the existing representation models that focus on the semantic information of individual clauses, our model breaks down the barriers of semantic association between clauses. The proposed event schema and integration schema use the event-based organization to share the semantic information of different clauses, and flexibly reorganize and accurately represent the semantic relations between clauses as well as the cross-code connections. Also, the construction and use sessions of DCKG are closer to human cognitive learning behavior, compared with the design code representation constructed by other methods. In addition, our model and method incorporate the "subgraphs-events-paths-triples" structure and event-based organization, which can record and provide feedback on the detailed checking results at different levels during the ACC process. Such good interpretability is important to promote the practical application of ACC (Soliman-Junior et al., 2021).

Existing research has largely ignored the exploration of the relation between clause semantic types and knowledge representation models (Zhang et al., 2022). Our study analyzes and extracts the semantic features of design code at subclause, clause, and code levels in conjunction with the need for ACC. Moreover, we propose the order, complex, event and integration schemas for different semantic types (unitary semantic with context, multiple semantic, nested semantic, Implicit semantic and logical relation, code organization, and reference relation). The relation between clause semantic types and knowledge representation is fully considered. The proposed DCKG representation model has higher representation ability compared to other models, which can represent more semantic types and a larger number of code clauses. Comparative experiments support this result.

As a domain knowledge application scenario, ACC requires high-quality knowledge from the representation model, similar to legislation and drug contraindications. However, there are no quality assurance standards and accepted processes for evaluating whether the actual modeling results of the representation are equivalent to the original code texts (Amor and Dimyadi, 2021). Compared with existing studies that only give examples to verify the feasibility of representation models, our method provides guarantees for the quality of DCKG in three aspects: 1) a clear and explicit semantic interpretation and knowledge reconstruction process; 2) the predefined domain expert intervention and semantic representation models; 3) the design code annotation platform and clause subgraph review mechanism. In addition, we have completed a DCKG corresponding to the *Check object* "*Equipment in station for the passenger*" mentioned in Fig. 15 with the annotation platform. The original data contains 20 clauses from different design codes. The constructed DCKG contains 159 nodes and 331 edges. Among them, |S|=331, |BS==89, |OS==11, |CS==21, |ES==106, |IS==102. The DCKG can be downloaded at (https://github.com/Yang-Mingsong/DCKG4ACC).

The model and method proposed in this research can also be extended to the formalized representation of domain knowledge with complex constrained documents such as legal, medical, mechanical design, and knowledge-driven fault diagnosis.

The main limitations of this research are identified and will be studied in the future by the authors. 1) Currently, we divide the design codes into process-oriented clauses and result-oriented clauses. Among them, the result-oriented clauses for ACC not only contain text format but also contains complex tables, verification formulas, appendices, Illustrations and other data formats. 2) To guarantee DCKG with high knowledge quality, the automatic level of construction process is limited. The final result of ACC should be a fully automated system. However, the current research requires a trade-off between the knowledge representation quality and automation level. The DCKG construction method proposed in this research still requires domain experts to intervene in the semantic interpretation and knowledge reconstruction processes. 3) This paper focuses on the first stage of ACC "*interpretation and representation of design codes*". The subsequent stages are only reflected in Section 5.2.4 as a case study, where the checking process and methods need to be optimized.

To resolve the above limitations, the following improvements and extensions can be pursued. 1) We will try to extract semantic information from semi-structured tables using wrappers (Nie et al., 2019) and study wrapper definition methods, automatic generation, updating and maintenance techniques for different table schemas. For verification formulas, appendices, diagrams and other special data formats, we consider upgrading the DCKG to a multi-modal knowledge graph to realize the design code representation of formula definitions, element descriptions, and diagram associations for ACC. 2) We can build a rich domain corpus based on the accumulated annotation data and expert templates. Then, we will combine deep learning models to implement various tasks of DCKG automated construction, such as entity recognition, entity linking, relation extraction, and semantic fusion. 3) We will use the DCKG as a domain knowledge base and delve into the IFC model enrichment for ACC, the execution and judgment of DCKG-based compliance checking, and the interpretation and feedback of checking results.

7. Conclusion

To enhance the expressiveness of design code representation for ACC, we propose a DCKG representation model and an accompanying semi-automatic construction methodology. First, we analyzed the semantic types contained in the design codes from three dimensions: subclause level, clause level and code level. Secondly, we have proposed a new model DCKG that can represent all semantic types. Four schemas are proposed into the new model. Among them, the order schema is proposed to improve the ability of DCKG to represent unitary semantic in subclauses. The complex schema is proposed to solve the problem of multiple and nested semantic representation in subclauses. The event schema is proposed to complete the representation of implicit semantic and logic in code clauses. The integration schema is proposed to improve the semantic representation and organization at the code level. Thirdly, we proposed an accompanying semi-automatic method comprising the well-defined semantic interpretation and knowledge reconstruction process. In addition, a design code annotation platform has been developed to implement the DCKG construction. The experimental results show that the DCKG representation model is of higher representation ability. The feasibility and correctness are validated with metro design code and station model as case study.

CRediT authorship contribution statement

Mingsong Yang: Conceptualization, Methodology, Writing – original draft, Writing – review & editing. **Qin Zhao**: Data curation, Funding acquisition, Project administration. **Lei Zhu**: Software, Writing – review & editing. **Haining Meng**: Writing – review & editing, Validation. **Kehai Chen**: Resources, Writing – review & editing. **Zongjian Li**: Investigation, Resources. **Xinhong Hei**: Supervision, Resources, Funding acquisition, Writing – review & editing.

Declaration of Competing Interest

The authors declare that they have no known competing financial interests or personal relations that could have appeared to influence the work reported in this paper.

Data Availability

I have shared the link to my data/code at the Attach File step.

Acknowledgment

The authors are grateful for the National Key R&D Program of China (2022YFB2602203), National Natural Science Foundation of China (51878556), Collaborative Innovation Center Project of Shaanxi Provincial Department of Education (20JY049), and Project of State Key Laboratory of Rail Transit Engineering Information (SKLKZ21-03).

References

Abu-Salih, B., 2021. Domain-specific knowledge graphs: a survey. J. Netw. Comput. Appl. 185, 103076 https://doi.org/10.1016/j.jnca.2021.103076.

Amor, R., Dimyadi, J., 2021. The promise of automated compliance checking. Dev. Built Environ. 5, 100039 https://doi.org/10.1016/j.dibe.2020.100039.

Arndt, D., Broekstra, J., DuCharme, B., Lassila, O., Patel-Schneider, P.F., Prud'hommeaux, E., Thibodeau, T., Thompson, B., 2021. December. RDF-star and SPARQL-star. W3C Community Group Draft Report. Retrieved April 3, 2022. ⟨https://www.w3.org/2021/12/rdf-star.html⟩.

Balaban, Ö., Kilimci, E.S.Y., Çağdaş, G., 2012. Automated code compliance checking model for fire egress codes. ECAADe 2 (1), 117–125. ⟨http://cumincad.scix.net/cgi-bin/works/Show?_id=ecaade2012_247⟩.

Beach, T.H., Rezgui, Y., Li, H., Kasim, T., 2015. A rule-based semantic approach for automated regulatory compliance in the construction sector. Expert Syst. Appl. 42 (12), 5219–5231. https://doi.org/10.1016/j.eswa.2015.02.029.

Beach, T.H., Hippolyte, J.L., Rezgui, Y., 2020. Towards the adoption of automated regulatory compliance checking in the built environment. Autom. Constr. 118, 103285 https://doi.org/10.1016/J.AUTCON.2020.103285.

Cao, J., Bucher, D.F., Hall, D.M., Eggers, M., 2022. A graph-based approach for module library development in industrialized construction. Comput. Ind. 139, 103659 https://doi.org/10.1016/J.COMPIND.2022.103659.

Clayton, M.J., Fudge, P., Jack Thompson, 2013, July. Automated plan review for building code compliance using BIM. In: Proceedings of the 20th International Workshop: Intelligent Computing in Engineering (EG-ICE 2013). ⟨https://www.researchgate.net/publication/254862600⟩.

Deng, H., Xu, Y., Deng, Y., Lin, J., 2022. Transforming knowledge management in the construction industry through information and communications technology: a 15-year review. Autom. Constr. 142, 104530 https://doi.org/10.1016/J.AUTCON.2022.104530.

Eastman, C., Lee, J., Jeong, Y., Lee, J., 2009. Automatic rule-based checking of building designs. Autom. Constr. 18 (8), 1011–1033. https://doi.org/10.1016/j.autcon.2009.07.002.

Ehrlinger, L., Wöß, W. 2016, September. Towards a Definition of Knowledge Graphs. Joint Proceedings of the Posters and Demos Track of 12th International Conference on Semantic Systems - SEMANTiCS2016 and 1st International Workshop on Semantic Change & Evolving Semantics (SuCCESS16). ⟨https://www.researchgate.net/publication/323316736⟩.

Fatemi, B., Taslakian, P., Vazquez, D., Poole, D. 2020. Knowledge Hypergraphs: Prediction Beyond Binary Relations. Proceedings of the Twenty-Ninth International Joint Conference on Artificial Intelligence, pp. 2191–2197. ⟨https://doi.org/10.24963/ijcai.2020/303⟩.

Fenves, S.J., 1966. Tabular decision logic for structural design. J. Struct. Div. 92 (6), 473–490. https://doi.org/10.1061/JSDEAG.0001567.

Gentile, A.L., Gruhl, D., Ristoski, P., Welch, S. 2019. Personalized Knowledge Graphs for the Pharmaceutical Domain (pp. 400–417). ⟨https://doi.org/10.1007/978-3-030-30796-7_25⟩.

Ghannad, P., Lee, Y.-C., Dimyadi, J., Solihin, W., 2019. Automated BIM data validation integrating open-standard schema with visual programming language. Adv. Eng. Inform. 40, 14–28. https://doi.org/10.1016/j.aei.2019.01.006.

Gottschalk, S., Demidova, E. 2018. EventKG: a multilingual event-centric temporal knowledge graph. In The Semantic Web. ESWC 2018. Lecture Notes in Computer Science (Vol. 10843, pp. 272–287). Springer. ⟨https://doi.org/10.1007/978-3-319-93417-4_18⟩.

Häußler, M., Esser, S., Borrmann, A., 2021. Code compliance checking of railway designs by integrating BIM, BPMN and DMN. Autom. Constr. 121, 103427 https://doi.org/10.1016/J.AUTCON.2020.103427.

Hjelseth, E., Nisbet, N. 2010. Exploring semantic based model checking. Proceedings of the 27th CIB W78 Conference, 27.

Hogan, A., Blomqvist, E., Cochez, M., D'amato, C., Melo, G., De, Gutierrez, C., Kirrane, S., Gayo, J.E.L., Navigli, R., Neumaier, S., Ngomo, A.-C.N., Polleres, A., Rashid, S.M., Rula, A., Schmelzeisen, L., Sequeda, J., Staab, S., Zimmermann, A., 2021. Knowledge Graphs. CoRRabs/2003 02320 (2021). https://doi.org/10.48550/arXiv.2003.02320.

Hu, Z.-Z., Leng, S., Lin, J.-R., Li,·Sun-Wei, Xiao, Y.-Q., 2022. Knowledge Extraction and Discovery Based on BIM: A Critical Review and Future Directions. 29, pp. 335–356. ⟨https://doi.org/10.1007/s11831-021-09576-9⟩.

Huang, H., Hong, Z., Zhou, H., Wu, J., Jin, N., 2020. Knowledge graph construction and application of power grid equipment. Math. Probl. Eng. 2020, 1–10. https://doi.org/10.1155/2020/8269082.

Jiang, L., Shi, J., Wang, C., 2022. Multi-ontology fusion and rule development to facilitate automated code compliance checking using BIM and rule-based reasoning. Adv. Eng. Inform. 51, 101449 https://doi.org/10.1016/j.aei.2021.101449.

Kejriwal, M., 2019. Domain-specific knowledge graph construction. Springe Int. Publ. https://doi.org/10.1007/978-3-030-12375-8.

Kerrigan, S.L., Law, K.H., 2005. Regulation-centric, logic-based compliance assistance framework. J. Comput. Civ. Eng. 19 (1), 1–15. https://doi.org/10.1061/(ASCE)0887-3801(2005)19:1(1).

Kim, H., Lee, J.K., Shin, J., Choi, J., 2019. Visual language approach to representing KBimCode-based Korea building code sentences for automated rule checking. J. Comput. Des. Eng. 6 (2), 143–148. https://doi.org/10.1016/J.JCDE.2018.08.002.

Kondreddi, S.K., Triantafillou, P., Weikum, G., 2013. HIGGINS. Proc. 22nd Int. Conf. World Wide Web 85–86. https://doi.org/10.1145/2487788.2487825.

Lee, Y.-C., Ghannad, P., Dimyadi, J., Lee, J.-K., Solihin, W., Zhang, J., 2020. A comparative analysis of five rule-based model checking platforms. Constr. Res. Congr. 2020, 1127–1136. https://doi.org/10.1061/9780784482865.119.

Li, L., Wang, P., Yan, J., Wang, Y., Li, S., Jiang, J., Sun, Z., Tang, B., Chang, T.-H., Wang, S., Liu, Y., 2020. Real-world data medical knowledge graph: construction and applications. Artif. Intell. Med. 103, 101817 https://doi.org/10.1016/j.artmed.2020.101817.

Li, X., Lyu, M., Wang, Z., Chen, C.-H., Zheng, P., 2021. Exploiting knowledge graphs in industrial products and services: a survey of key aspects, challenges, and future perspectives. Comput. Ind. 129, 103449 https://doi.org/10.1016/j.compind.2021.103449.

Lin, J., Zhao, Y., Huang, W., Liu, C., Pu, H., 2021. Domain knowledge graph-based research progress of knowledge representation. Neural Comput. Appl. 33 (2), 681–690. https://doi.org/10.1007/s00521-020-05057-5.

Ma, X., 2022. Knowledge graph construction and application in geosciences: a review. Comput. Geosci. 161, 105082 https://doi.org/10.1016/j.cageo.2022.105082.

Macit İlal, S., Günaydın, H. M, 2017. Computer representation of building codes for automated compliance checking. Autom. Constr. 82, 43–58. https://doi.org/10.1016/j.autcon.2017.06.018.

Malsane, S., Matthews, J., Lockley, S., Love, P.E.D., Greenwood, D., 2015. Development of an object model for automated compliance checking. Autom. Constr. 49, 51–58. https://doi.org/10.1016/j.autcon.2014.10.004.

Nawari, N.O., 2019. A generalized adaptive framework (GAF) for automating code compliance checking. Buildings 9 (4), 86. https://doi.org/10.3390/buildings9040086.

Nie, Q., Sheng, X., Zhang, N., Wang, J., Shang, Y., Hu, F., 2019. Construction of a nautical knowledge graph based on multiple data sources. J. Coast. Res. 94 (sp1), 223. https://doi.org/10.2112/SI94-047.1.

Noy, N., Gao, Y., Jain, A., Narayanan, A., Patterson, A., Taylor, J., 2019. Industry-scale knowledge graphs. Commun. ACM 62 (8), 36–43. https://doi.org/10.1145/3331166.

Paulheim, H., 2016. Knowledge graph refinement: a survey of approaches and evaluation methods. Semant. Web 8 (3), 489–508. https://doi.org/10.3233/SW-160218.

Paulheim, H. 2018, October 8. How much is a Triple? Estimating the Cost of Knowledge Graph Creation. Proceedings of the ISWC 2018 Posters & Demonstrations, Industry and Blue Sky Ideas Tracks Co-Located with 17th International Semantic Web Conference (ISWC 2018). ⟨http://ceur-ws.org/Vol-2180/ISWC_2018_Outrageous_Ideas_paper_10.pdf⟩.

Pauwels, P., Zhang, S., 2015. Semantic rulechecking for regulation compliance checking: an overview of strategies and approaches. In: Proceedings of the 32nd CIB W78 Conference, pp. 619–628. ⟨https://www.mendeley.com/catalogue/96a4a44a-8b01-345e-a731-bcb48367b234/⟩.

Pauwels, P., van Deursen, D., Verstraeten, R., de Roo, J., de Meyer, R., van de Walle, R., van Campenhout, J., 2011. A semantic rule checking environment for building performance checking. Autom. Constr. 20 (5), 506–518. https://doi.org/10.1016/j.autcon.2010.11.017.

Pingle, A., Piplai, A., Mittal, S., Joshi, A., Holt, J., Zak, R. 2019. RelExt: Relation Extraction using Deep Learning approaches for Cybersecurity Knowledge Graph Improvement. Proceedings of the 2019 IEEE/ACM International Conference on Advances in Social Networks Analysis and Mining, pp. 879–886. ⟨https://doi.org/10.1145/3341161.3343519⟩.

Schwabe, K., Teizer, J., König, M., 2019. Applying rule-based model-checking to construction site layout planning tasks. Autom. Constr. 97, 205–219. https://doi.org/10.1016/J.AUTCON.2018.10.012.

Singhal, A. 2012, May 16. Introducing the Knowledge Graph: things, not strings. ⟨https://blog.google/products/search/introducing-knowledge-graph-things-not/⟩.

Solibri. 2021. Using External Data in Rule Parameters – Solibri Desktop Help Center. ⟨https://help.solibri.com/hc/en-us/articles/4416661518871-Using-External-Data-in-Rule-Parameters⟩.

Solihin, W., Eastman, C., 2015. Classification of rules for automated BIM rule checking development. Autom. Constr. 53, 69–82. https://doi.org/10.1016/J.AUTCON.2015.03.003.

Solihin, W., Eastman, C., 2016. A knowledge representation approach to capturing bim based rule checking requirements using conceptual graph. CIB W78 2015 Spec. Track Compliance Checking 21, 370–401. ⟨http://www.itcon.org/2016/24⟩.

Soliman-Junior, J., Tzortzopoulos, P., Baldauf, J.P., Pedo, B., Kagioglou, M., Formoso, C. T., Humphreys, J., 2021. Automated compliance checking in healthcare building design. Autom. Constr. 129, 103822 https://doi.org/10.1016/j.autcon.2021.103822.

Stoica, R., Fletcher, G., Sequeda, J.F., 2019. On directly mapping relational databases to property graphs. In: Hogan, A., Milo, T. (Eds.), Alberto Mendelzon Workshop on Foundations of Data Management (AMW2019). CEUR-WS.org. ⟨http://ceur-ws.org/Vol-2369⟩.

Tan, X., Hammad, A., Fazio, P., 2010. Automated code compliance checking for building envelope design. J. Comput. Civ. Eng. 24 (2), 203–211. https://doi.org/10.1061/(ASCE)0887-3801(2010)24:2(203).

Tang, M., Su, C., Chen, H., Qu, J., Ding, J., 2020. SALKG: A Semantic Annotation System for Building a High-quality Legal Knowledge Graph. 2020 IEEE International

Conference on Big Data (Big Data), 2153–2159. ⟨https://doi.org/10.1109/BigData50022.2020.9378107⟩.
Wang, J., Mu, L., Zhang, J., Zhou, X., Li, J., 2020. On intelligent fire drawings review based on building information modeling and knowledge graph. Constr. Res. Congr. 2020, 812–820. https://doi.org/10.1061/9780784482865.086.
Wang, Q., Wang, H., Lyu, Y., Zhu, Y., 2021. Link prediction on n-ary relational facts: a graph-based approach. Find. Assoc. Comput. Linguist.: ACL-IJCNLP 2021, 396–407. https://doi.org/10.18653/v1/2021.findings-acl.35.
Wen, J., Li, J., Mao, Y., Chen, S., Zhang, R., 2016. On the representation and embedding of knowledge bases beyond binary relations. Proc. 25th Int. Jt. Conf. Artif. Intell. 1300–1307. In: ⟨https://www.ijcai.org/Proceedings/16/Papers/188.pdf⟩.
Xu, X., Cai, H., 2020. Semantic approach to compliance checking of underground utilities. Autom. Constr. 109, 103006 https://doi.org/10.1016/j.autcon.2019.103006.
Xue, X., Zhang, J., 2021. Part-of-speech tagging of building codes empowered by deep learning and transformational rules. Adv. Eng. Inform. 47, 101235 https://doi.org/10.1016/j.aei.2020.101235.
Yang, Q.Z., Xu, X., 2004. Design knowledge modeling and software implementation for building code compliance checking. Build. Environ. 39 (6), 689–698. https://doi.org/10.1016/j.buildenv.2003.12.004.
Zhang, J., El-Gohary, N.M., 2017. Semantic-based logic representation and reasoning for automated regulatory compliance checking. J. Comput. Civ. Eng. 31 (1), 04016037. https://doi.org/10.1061/(ASCE)CP.1943-5487.0000583.
Zhang, R., El-Gohary, N., 2021. A deep neural network-based method for deep information extraction using transfer learning strategies to support automated compliance checking. Autom. Constr. 132, 103834 https://doi.org/10.1016/j.autcon.2021.103834.
Zhang, Y., Sheng, M., Zhou, R., Wang, Y., Han, G., Zhang, H., Xing, C., Dong, J., 2020. HKGB: an inclusive, extensible, intelligent, semi-auto-constructed knowledge graph framework for healthcare with clinicians' expertise incorporated. Inf. Process. Manag. 57 (6), 102324 https://doi.org/10.1016/j.ipm.2020.102324.
Zhang, Z., Ma, L., Broyd, T. 2022, July 24. Towards fully-automated code compliance checking of building regulations: challenges for rule interpretation and representation. ⟨https://doi.org/10.35490/EC3.2022.148⟩.
Zhong, B., Ding, L., Luo, H., Zhou, Y., Hu, Y., Hu, H.M., 2012. Ontology-based semantic modeling of regulation constraint for automated construction quality compliance checking. Autom. Constr. 28, 58–70. https://doi.org/10.1016/j.autcon.2012.06.006.
Zhong, B., Gan, C., Luo, H., Xing, X., 2018. Ontology-based framework for building environmental monitoring and compliance checking under BIM environment. Build. Environ. 141, 127–142. https://doi.org/10.1016/j.buildenv.2018.05.046.
Zhou, Y.C., Zheng, Z., Lin, J.R., Lu, X.Z., 2022. Integrating NLP and context-free grammar for complex rule interpretation towards automated compliance checking. Comput. Ind. 142, 103746 https://doi.org/10.1016/J.COMPIND.2022.103746.